INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

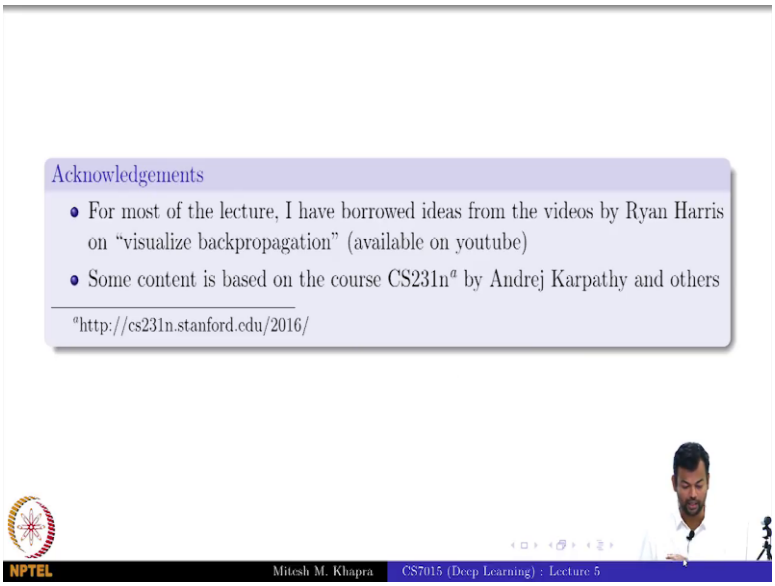
Lecture - 05
Gradient Descent (GD), Momentum Based GD, Nesterov Accelerated GD, Stochastic GD, AdaGrad, RMSProp, Adam

Welcome to lecture 5 of the course on Deep learning. And so, today we look at some variants of gradient descent. So, we will just quickly do a recap of gradient descent and then look at some variants of it, or some ways of improving it, which is momentum based gradient descent, Nesterov of accelerated gradient descent, stochastic gradient descent, AdaGrad RMSProp and Adam.

So, just to set the context. So, we started with this gradient descent algorithm for a single sigmoid neuron, and then we saw how to extend to network of neurons with back propagation. So, we realized that all we need is the gradients or the partial derivatives, with respect to all the weights and biases. Once we compute that we can just use the gradient descent update rule.

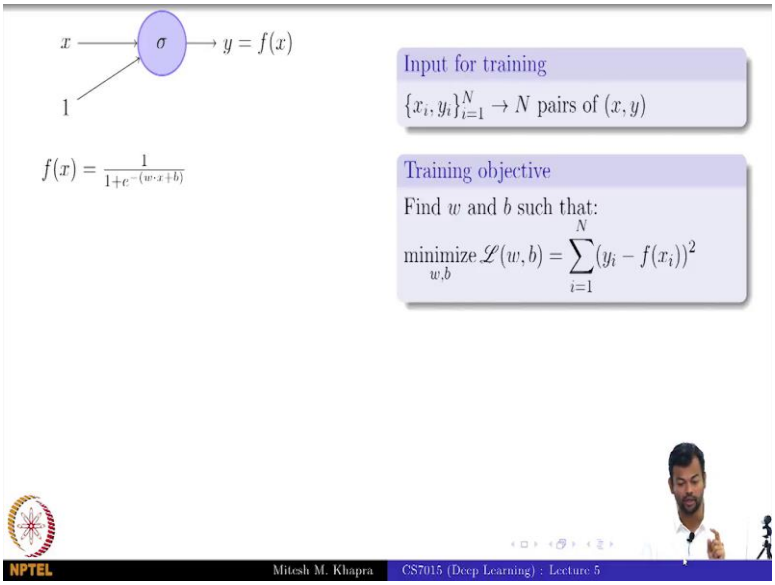
Now, today what we are going to see is, are there better update rules which lead to faster conversion or better performance in various ways. So, that is why we are going to look at all these different variants or methods of improving on gradient descent. So, that is the context.

(Refer Slide Time: 01:18)



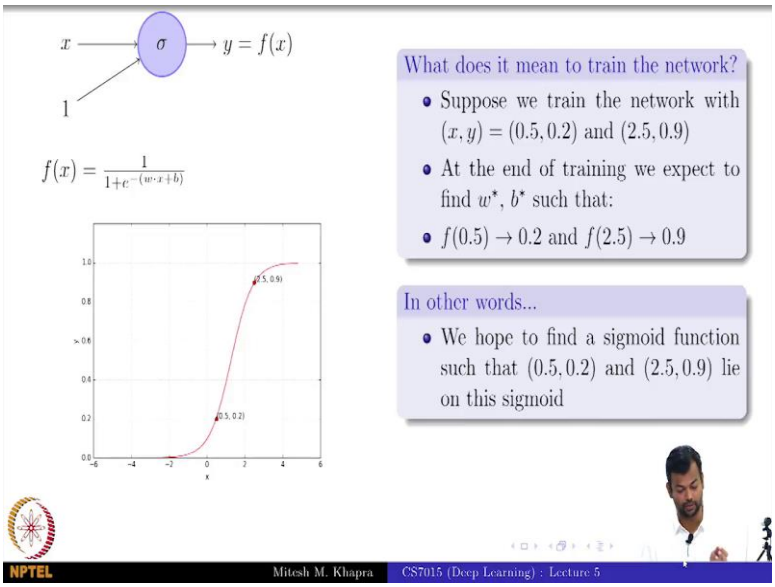
I will just quickly rush through. So, for most of the lecture, I have borrowed ideas from the videos by Ryan Harris on visualize back propagation and some content is based on this course by Andrej Karpathy and others, when I talk about some tips for learning rate and so on. So, you can just look at those also. So, we will just quickly rush through the first two modules which we have already done.

(Refer Slide Time: 01:46)

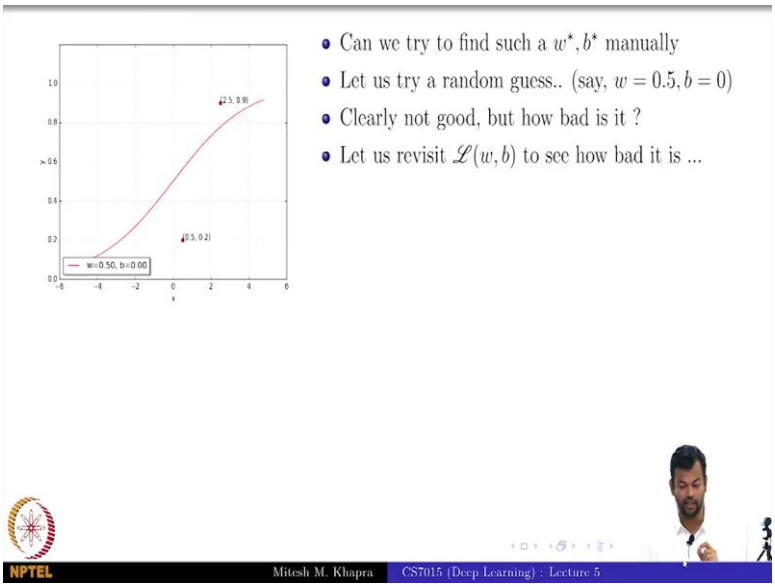


Which was, we were interested in learning the weights and biases for this very toy network, with just 1 input and 1 output, and we started by doing something known as guesswork where we were just trying to adjust these weights and biases by hand.

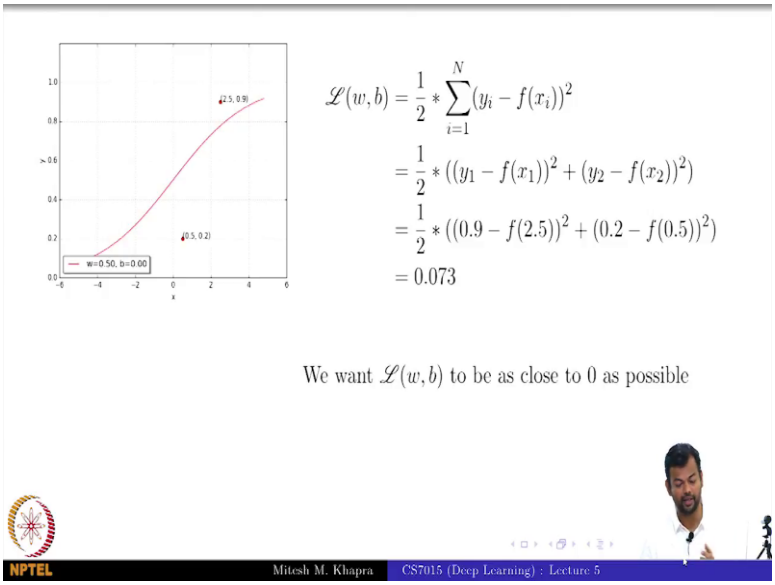
(Refer Slide Time: 01:52)



(Refer Slide Time: 01:56)

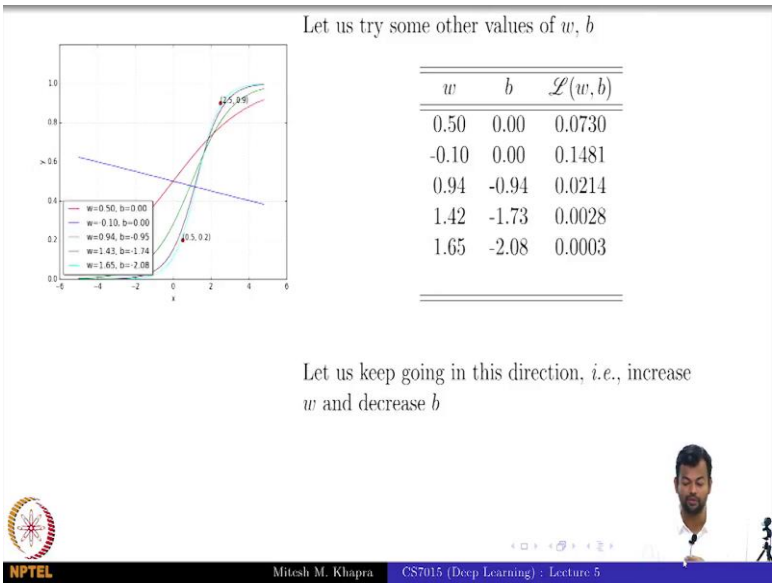


(Refer Slide Time: 01:58)

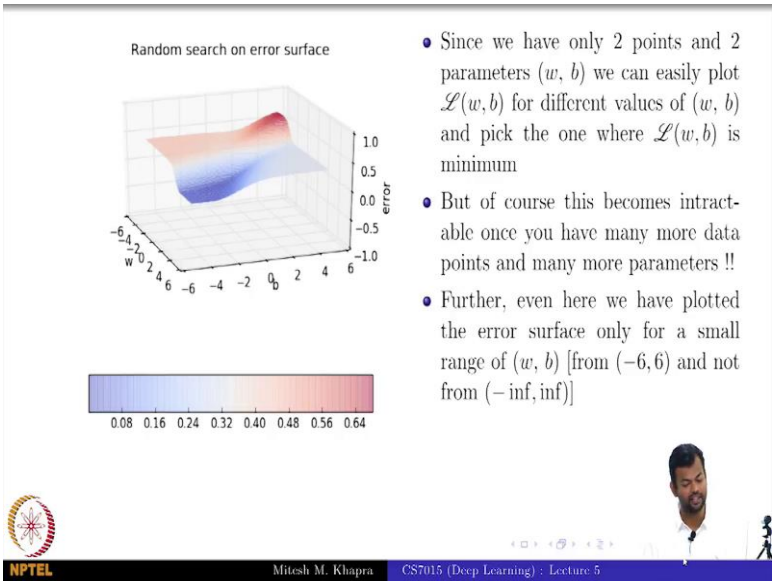


And we realized that its clearly not good and, but we still try to do a very smart guess work, where we were driven by this loss function, which was telling us whether this guess, the current guess is better than the previous guess or not. And we just kept following our guess work and try to reach to some solution, and for this toy network it was very easy to do that.

(Refer Slide Time: 02:06)

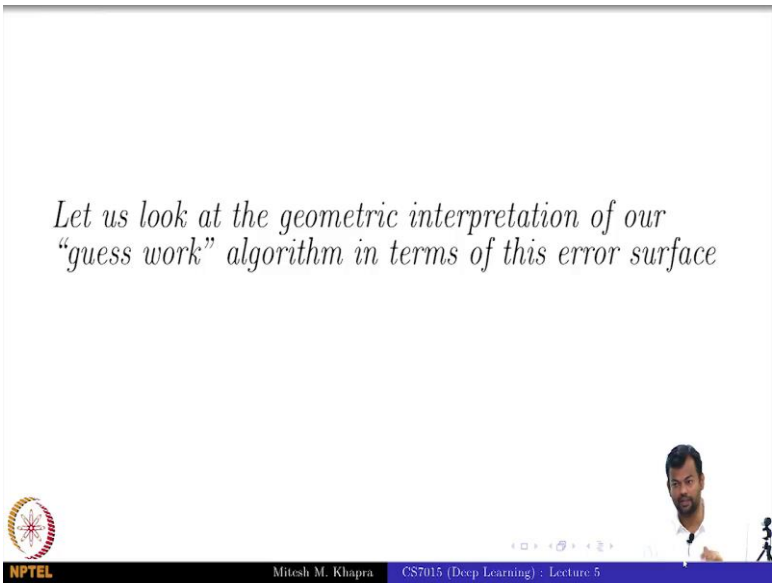


(Refer Slide Time: 02:17)



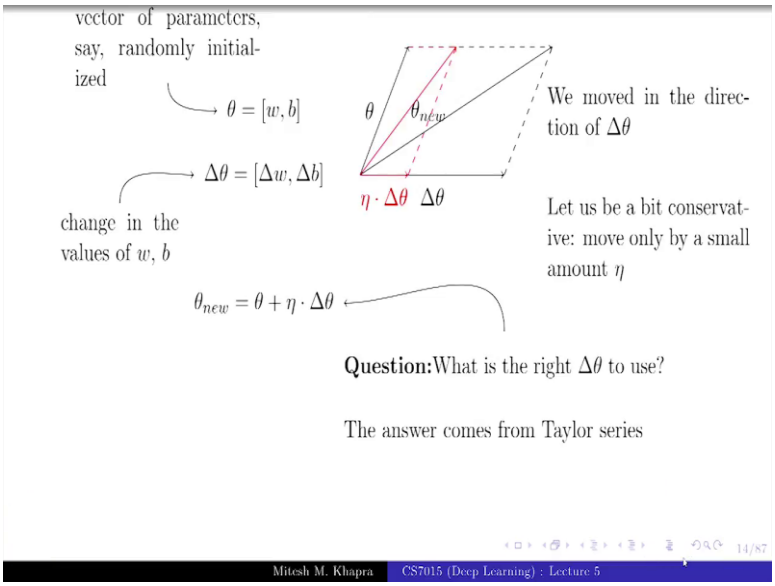
And what we were actually doing is, there is this error surface which exists, which can be plotted for all possible values of w comma b. And what we were trying to do with this guesswork is, trying to find path over the error surface, so that we enter into the better regions. So, red is bad, blue is good; the darker the shade of blue the better. And this of course, becomes intractable when you have many parameters and so on.

(Refer Slide Time: 02:38)



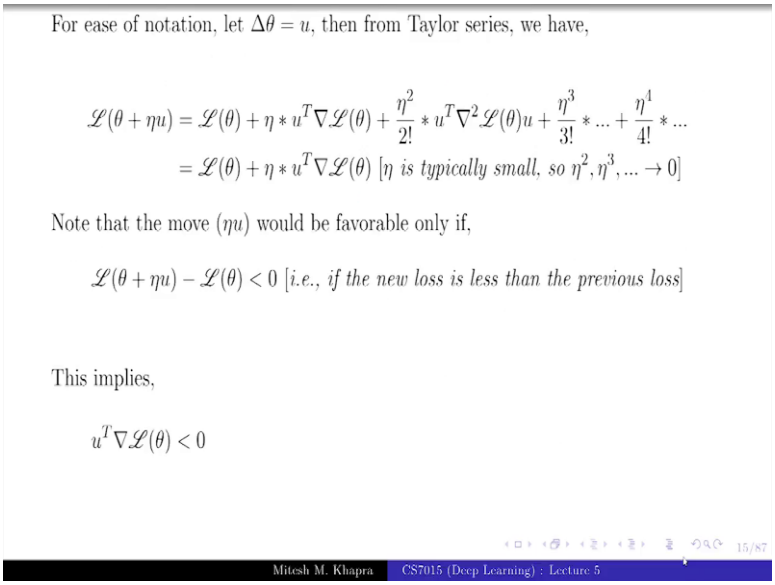
So, we wanted to have a better way of navigating the error surface. So, this is exactly what we were doing with the guesswork algorithm.

(Refer Slide Time: 02:48)



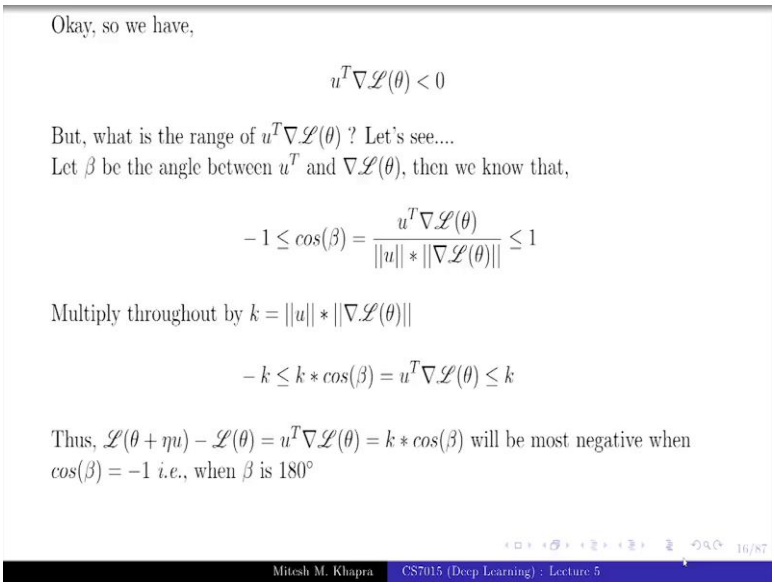
So, then this better way actually we realized that we could arrive at it from a very principled solution from, starting from Taylor series.

(Refer Slide Time: 02:55)

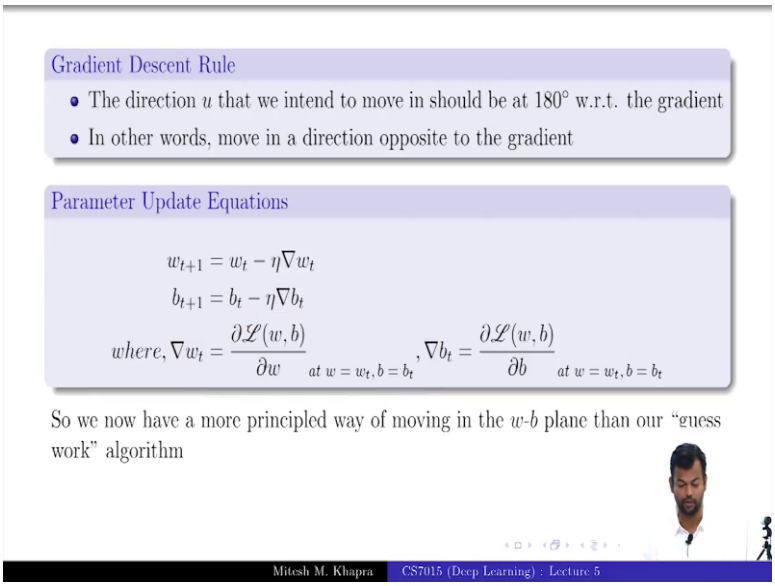


And we went to this derivative, where we finally came up with this rule that move in the direction opposite to the gradient.

(Refer Slide Time: 02:58)

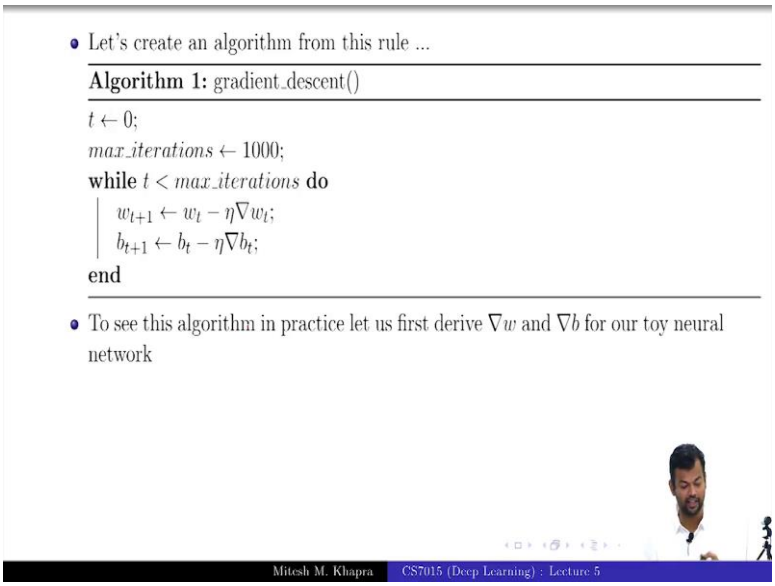


(Refer Slide Time: 03:03)



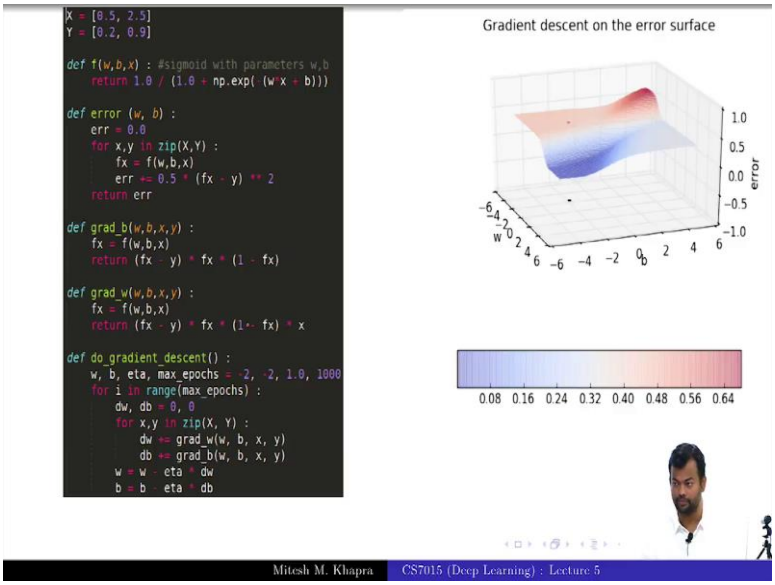
So, that is the rule that we have been sticking to since then. And we also along the way realize some of these things which we defined carefully which was, what is, what exactly this quantity means, which is the partial derivative with respect to w evaluated at a particular weight comma bias configuration. And because this is an iterative process, you are at a certain value of weight and bias and you need to change it from there.

(Refer Slide Time: 03:30)



And we then created an algorithm out of this and when we ran this, we actually derived the full derivative and so on.

(Refer Slide Time: 03:38)



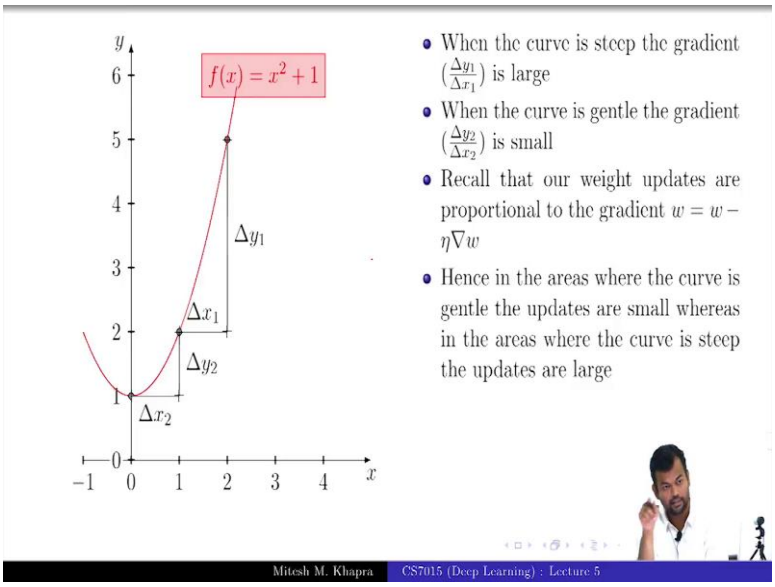
And then when we finally, ran this algorithm. So, this is where, now I will slow down. So, when we ran this algorithm. So, let us see what was happening here right. So, I will just start the algorithm from the beginning.

So, we are now going to run this code and you tell me something that you observe ok. So, I am just clicking. So, there is no change in the pace at which I am clicking this

right? So, every click of this is one time step and I am just continuously clicking this I will start now, do you observe something [FL] ok. Do you observe something?

It was initially slow then suddenly picked up and then it again became slow. Why did this happen? The slope is small why ok. How many of you completely understand why this slow and fast moment was there, please raise your hands good. So, that is what we will focus on now right? So, we will try to see this.

(Refer Slide Time: 04:30)



So, we will, I hope this has been fixed ok. So, let us take a simple function which is f of x equal to x square plus 1 right, this is how it will look like. Now in these portions of the curve, the curve is actually very steep right and in these portions the curve is a bit gentle and of course, it becomes very gentle over here right? All of you can see the pen marks properly.

So, now let us see what this means; this steep and fast and small. So, let us look at a region which is steep ok. Now what I am going to do is, I am going to change my x by 1, I move my x from 1 to 2. How much did my y change. All you need to do is just substitute in this formula right for 2 it evaluates to 5, for 1 it evaluates to 2. So, when you move from 1 to 2, your function changed from 2 to 5. Ok so, there is a large change in the function for 1 unit change in your value of x, everyone sees that.

Now, let me do the same at a gentle portion of the curve, I will do it here. Now when I changed the x by 1 unit, again 1 unit right, it is the same change which I did earlier. I changed from zero to 1, how much did my y change.

Student: 1.

1.ok Now actually what is this quantity; delta y 1 by delta x 1.

Student: Slope.

It is the slope, it is the derivative at that point. So, what are you inferring from this. What happens to the derivative when you are at steep slopes.

Student: It is high.

Derivative is high, because the change in y is much faster than the change in x. What happens to the derivative when you are at the gentle slopes.

Student: Smaller.

Smaller, because the change in y is small or relatively smaller as compared to the change in x or it could also be missing, but just these two are relatively different, is what I am trying to impress upon right. And so; that means, the derivatives at the steep slopes are larger in magnitude, whereas, for the gentle slopes they are smaller in magnitude.

Now, can you relate it to the observation that you had on the previous slide. When we were at the plateau it was a very dash slope, gentle slope what would the derivatives be

Student: Small.

Small now what are our updates, you have w is equal to w minus the derivative right? Now the derivative is small what will happen to the updates.

Student: Small.

They will be small. What would happen if the derivative is large.

Student: The updates would be large.

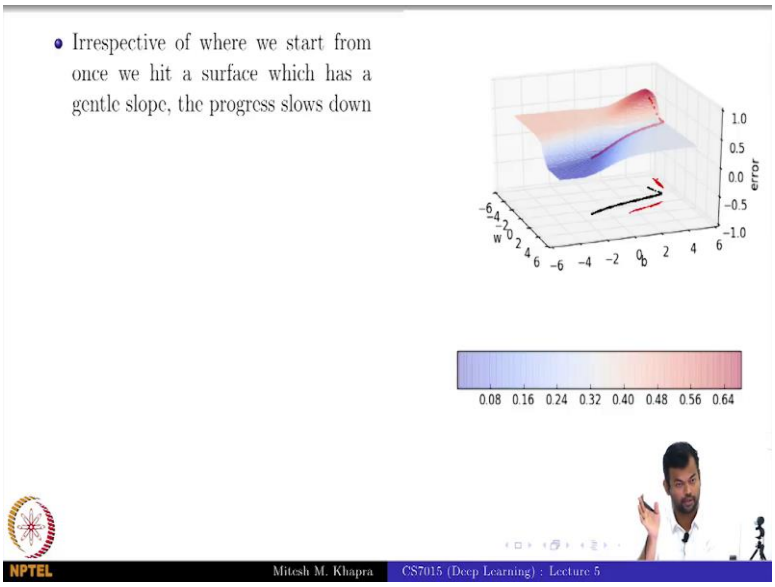
The updates would be large. Therefore, in the gentle areas you are moving slowly and in the steep areas you are moving fast. You get this picture very clearly. Now this is going to be the basis of a lot of things that we do today. So, it is very essential to that you understand this perfectly. All of you get this properly good.

(Refer Slide Time: 07:32)



Now, now you might say that this was only that special point again and I always get those questions. So, let us see what happens, if you start from a different point.

(Refer Slide Time: 07:40)



So, now again the same gradient descent algorithm I am going to run, but instead of starting at this point which was my random initialization, I just happened to choose a very different random initialization which is here, everyone sees that.

Now, let us see what happens, what do you expect initially fast movement, because the steep, the slope is a bit steep. Now what would happen? It will become slow because you have entered a gentle slope region and then again fast right. So, and then again it will become slow.

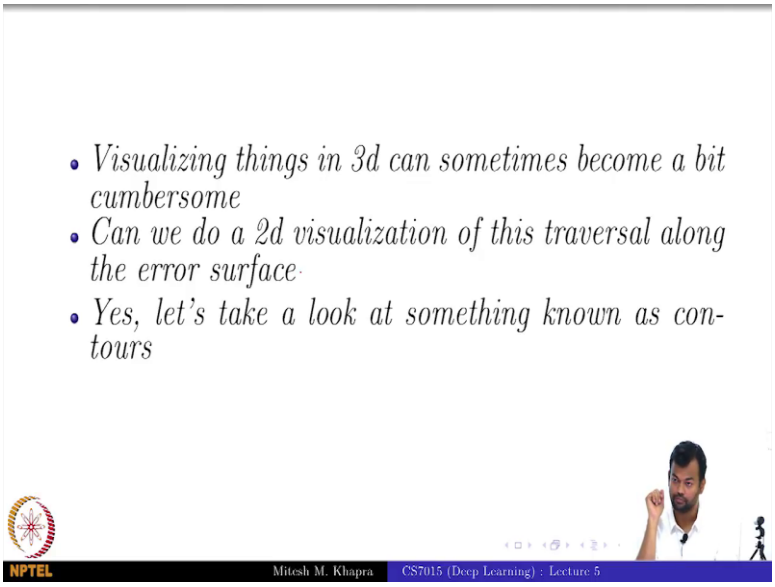
So, see in this gentle region right, the changes in w are so small that all your black points are actually indistinguishable from each other, it is almost like a snakes body whereas, in these steep slopes, you can see a large change in the w. You can see gaps between the values of w right? So, this is irrespective of where you start from. Gentle means slow movement, steep means fast movement that is the basis.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 5.3
Lecture - 05
Contours Maps

So we look at something known as Contours.

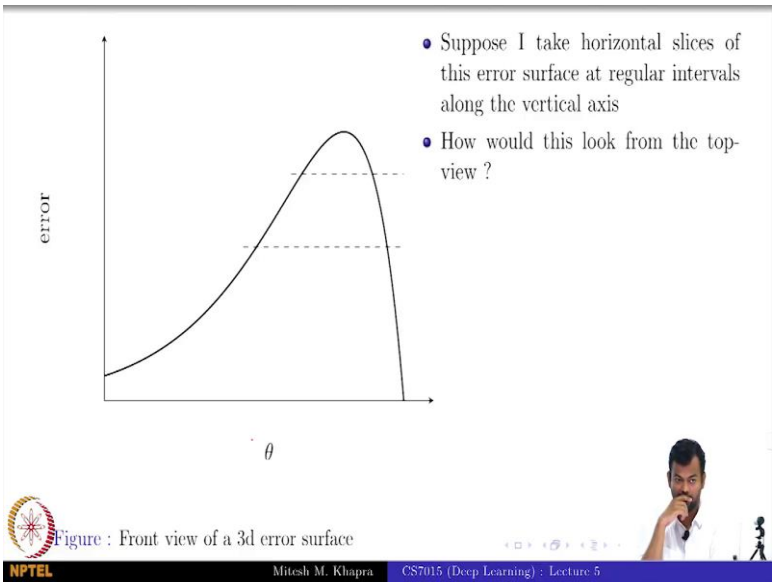
(Refer Slide Time: 00:16)



So, now visualizing things in 3d can sometimes become a bit difficult, especially for the person who is making the slides. So, we can, can we do a 2d visualization of this traversal. Have I done this in the ML course, no good? Can we do a 2d visualization of this traversal along the error surface.

So, for that we need to understand something known as contours. How many of you have looked at contour diagrams before, how many of you know how to read them. All of you know how to read them.

(Refer Slide Time: 00:47)



So, let us see, now suppose this is what my error surface looks like and I have a single scalar variable. So, this is just a function of w for example, and this is what my error surface looks like.

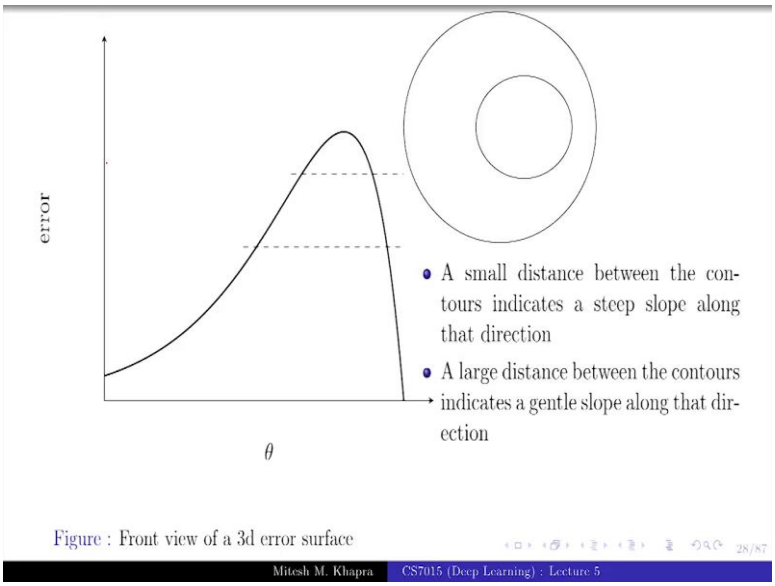
Now, what I am going to do is, I am going to take horizontal slices on this error surface fine. Now can you tell me how this is going to look from the top. Sorry let me, you should start answering before understand the question. This is this error surface is actually. So, I was wrong in saying this is theta, assume this is w comma b and you are just seeing the front view of the error surface. What you are seeing here is just the front view.

This error surface is actually like a dementors hat, so right. So, imagine that it is a hat place like this and you are just seeing the front view of this; otherwise a top view does not make sense right. So, now, I am going to slice this hat at two vertical positions, and now you are looking at it from the top, what are you going to see?

Student: Ellipsis.

Ellipsis, everyone agrees with that.

(Refer Slide Time: 01:49)



So, we will see something like this. do you see something peculiar about this. Is this a contour map. Is this? no ok, and all of you raise your hands when I asked do you know contour. So, do you see something peculiar about this, what is it? how many if you get that. So, what you are seeing here is, this portion right where the slope was very steep, the difference between the two circles or the two ellipses is small and you can visualize it if you try to look at it from the top, this distance is actually going to be small right, and in the areas where the slope was gentle, relatively gentle the distance is more and you can again visualize it right, from if you look at from the top this is the distance that you are going to see. And what do you say about these guys, what does that indicate. They are the same.

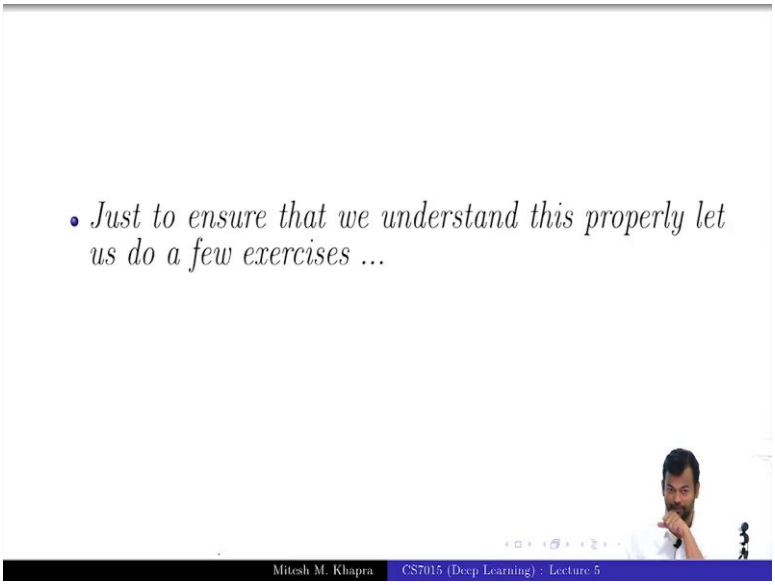
Student (Refer Time: 02:46).

Value, across that entire region the value is same, because you have taken a verticals, you have taken a horizontal slice at a particular vertical position right. So, you have taken a horizontal slice at this position; that means, the error is going to remain the same throughout that rim. Is this clear to everyone ok? It is very important that you understand this.

So, there are only two things that you need, to understand if you want to read contour maps; one is a small distance between the contours indicates that the steep slope exists along that direction, and a large distance between the contours indicates that a gentle

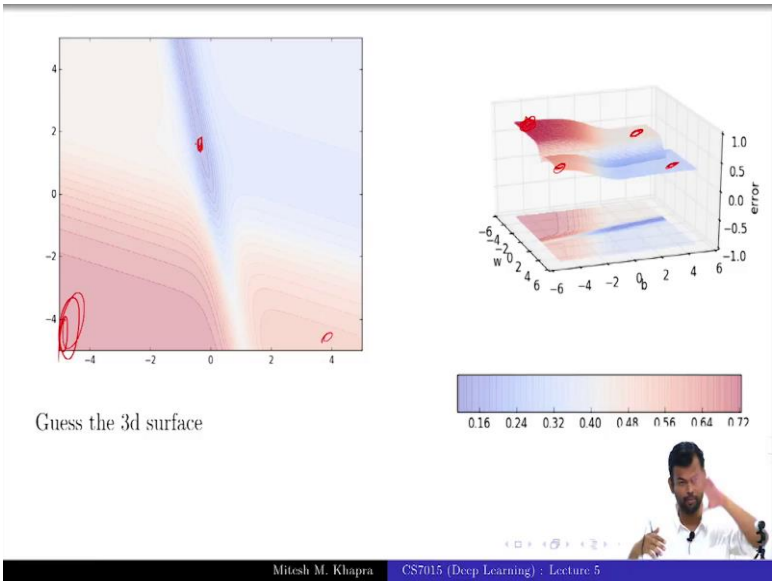
slope exists along the direction. So, everything today is going to be about steep and gentle slopes. And the other thing that you know need to know is that whenever you see one circle, the error is the same along that circle or ellipse, whatever you boundary that you see, the error is the same, because you are taking these vertical slices. So, we are ready with this rule, everyone understands this perfectly.

(Refer Slide Time: 03:40)



So, I will just give you a couple of exercises and you have to tell me whether you understand this or not.

(Refer Slide Time: 03:50)



So, I have plotted a 3 d surface or 2 d, I have, what is this.

Student: (Refer Time: 03:54).

No.

Student: there is a contour.

There is a contour; everything is not going to look like clean circles always right ok. So, this is a contour, every line that you see here represents one cut along the vertical axis right; that means, the error is the same there. Now what you are seeing is a contour, I want you to guess the 3 d surface from this, you just guess it, I mean just keep it to yourself fine. The color is the same right blue is good, red is bad.

So, blue means the darker, the shade of blue the lesser the value of the error, the darker the shade of red the higher the value of the error ok. I want you to imagine the 3 d surface, if you can do that then I will be sure that you understand what how to read a contour. How many if we can imagine this? You can just say yes right I can never figure out whether you actually speak it.

So, let me help you with the first one and then we will do a few more. So, let us start with the extremes right? So, let me see how to do this; so this portion. I also need to do it for the video ok. So, let me just do it here. So, this portion, what do you think about the slope there, very flattish why, because this is the line that you see and the other line is not even in the figure right? So, it is basically very flat, the slope is very gentle. Is it a low region or a high region, high region fine ok. Now what is actually happening here, what is the slope here

Student: High.

Very high; that is why these two regions are very close to each other. So, from this high region what is happening, suddenly there is a slope and you are going down and you know you are going down, because you are reaching a blue region right ok. Now what is` happening here?

Student: Very flat.

Very flat, and this also flat, but slightly upper than the lower guy; is that fine? Now can you all imagine this ok, and is this what you thought it is? Perfectly yes right is exactly what you thought ok. Just a minute. So, the orientation here has been changed a bit right. So, this portion actually corresponds to this portion. Are the two this is clear, this portion corresponds to this portion right the just orientated fine?

So, you start off this high plateau region which is here, then you start going down go down and then you see a fold here right; that is this fold. So, you went to a darker shade and then you came up to a slightly lighter shade. The shades are ok.

Guess the 3D surfaces. How many if you want to play this forever now. Start with the extremes the bad guys the good guys the plateaus and the valleys and then see how do you go from the plateau to the valley ok. Tell me the corners first, this plateau or valley.

Student: Plateau.

Plateau. This plateau, higher than this or lower than this.

Student: Lower than this.

Lower than this. This?

Student: Valley.

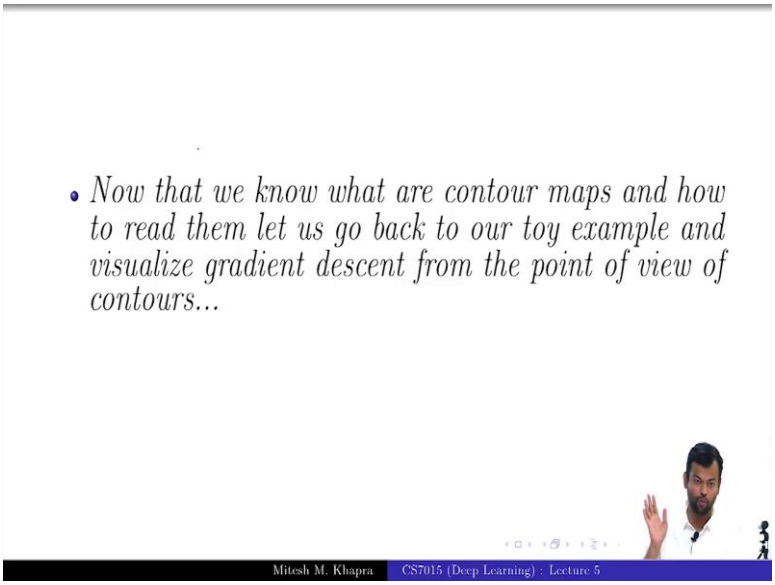
This, towards the valley, it is still between red and blue right, it is not like right down there and what happens to all these guys, all are very steep slows, all converging down into the valley. So, can you perfectly imagine this?

And you will tell, yes when I say when I show you the 3 d surface right, again you need to reorient yourself. So, this corner here is this corner, this corner here is this corner. So, we had these two plateaus at the top, we had this slightly higher valley slightly lower valley and then all of them going into a very deep valley. You see that. Everyone gets this. How many? If you have a problem with this if you have a problem with this, you will just sleep off in the rest of the lecture. So, I want you all to understand this very carefully I do not mind repeating it. How many of you understand this. You understand the regions with gentle slope

Student: Yes.

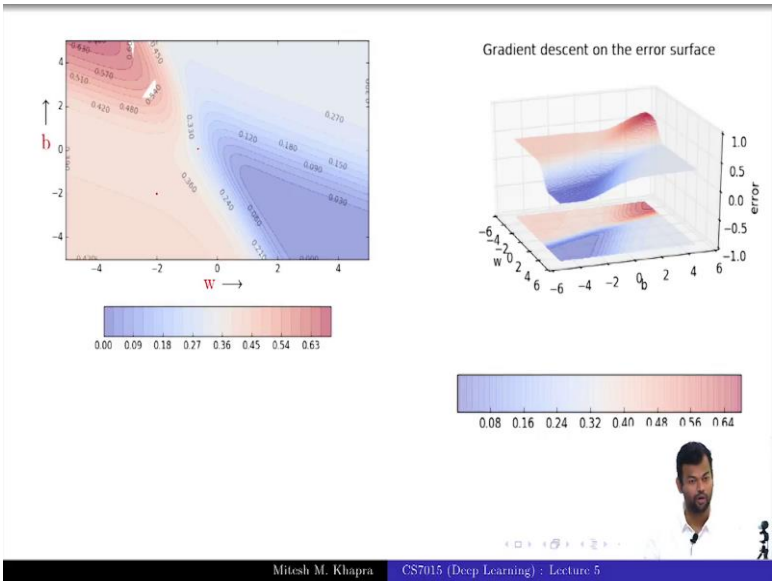
The regions where you have a steep slope and you end up into that valley, which is the valley here can you point it out, fine ok. So, we will move ahead.

(Refer Slide Time: 08:10)



So, now, we know what contour maps are and how to visualize them and so on right. So, now, we will try to see the gradient a descent algorithm. Instead of running it on the 3D error surface, we will try to run on this 2D contour map.

(Refer Slide Time: 08:23)



So, this is what I already showed you right? I started from here and I showed you how it comes here or something like this right? That was the gradient descent, let me just erase this ok, that is something like what the gradient descent algorithm.

Now, again you just need to reorient yourself. So, let us see, this corner is this corner, this corner is this corner and so on right? So, you get the reorientation right, it just shifted. Now I am going to start my gradient descent algorithm from here, from this point ok, everyone see is that ok. I am going to start from there and you have to help me and I am not going to just keep clicking, you have to tell me what is going to happen. So, what will happen initially, fast movement slow movement?

Student: Slow movement.

Slow movement right. So, I am running it 1 2 3 4 5 6 7 8, it just keeps running very slowly. Now what will happen?

Student: Fast.

Fast ok; now you see actually you can see the arrows these arrows are the quantity, the magnitude of the movement right. So, earlier this movement was so small that you could not even see the arrows, I have been drawing arrows right from the beginning, but you could not see them at the beginning, now you can see them right? Now what will happen?

Student: Slow.

Slow right; so, you see the exact same movement that I did on the 3 d surface, now you can visualize it on the 2 d surface right and you can easily tell me where it will go fast, where it will go slow right and where it will just keep moving very drag its feet and so on ok. So, this is where it starts dragging its feet, and the same thing happened when we were in this region right? So, just, you just make the connection that we are in the corresponding 3 d region there, ok fine. So, we are moving very slow and it just keeps running.

So, that is where we lend this module. So, we just revised gradient descent. We saw that things are proportional to the gradient that is why gradient descent, and the smaller the

gradient the slower the movement, the larger the gradient higher the movement, gentle the slope.

Student: Smaller.

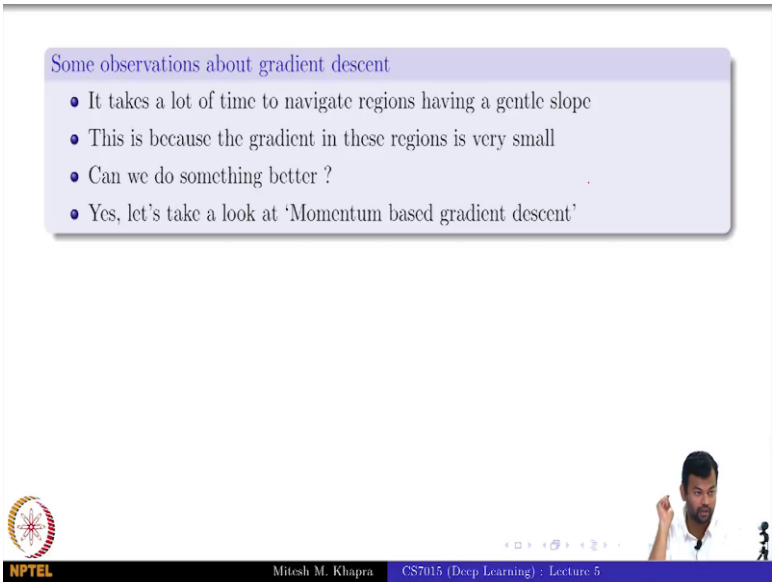
Smaller the gradient, steeper the slope, larger the gradient.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 5.4
Lecture – 05
Momentum Based Gradient Descent

In this module we will look at momentum based gradient descent.

(Refer Slide Time: 00:19)



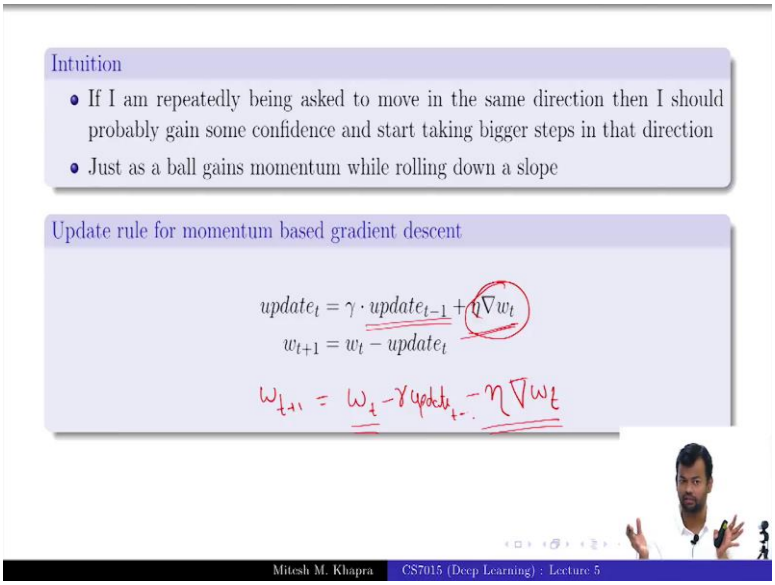
So, what were the observations about gradient descent that, it takes a lot of time to navigate regions having a gentle slope. So, what is the practical implication of this? in practice why it what does this need to, what does this mean right, it takes more time. So, remember we had said this max iteration equal to 1000.

Now, if you are initialization happens to be such that you are stuck in this large flat region, then those 1000 iterations just keep moving around that flat region right? You will not enter into one of the valleys and valleys is what you are interested in right, because values is where you will have some minima for your function right?

So, if you have a very gentle slope, then for 1000 iterations you will keep moving around that gentle slope right, that is why this has a practical implication. Now this was because

the gradient in these regions were small, can we do something better that is the question right? So, yes we can and we will take a look at momentum based gradient descent.

(Refer Slide Time: 01:14)



So, here is the analogy which I give, my TA's have heard this at least 10 times. So, I will just repeat it the 11 time for them. So, I hope that is the one which I want to use here yeah ok. So, now, suppose you are standing at the Velachery gate and you want to go to Phoenix market city, something that all if you can relate to today. So, you want to go to Phoenix market city and you ask the security guy at the gate that where do I go right?

So, he will say take a left, no take a right? So, I am slightly dyslexic, actually I have a left right dyslexia. So, take, take a right ok. So, you will say he has told me to move right, but you would still be a bit cautious right, we will just keep moving slowly in that direction, that is how we find ask for directions, you keep moving slowly in that direction?

Now, 100 steps later or 100 meters later you find another guy and you ask him or her where is phoenix market city. He again points to in the same direction, keep moving left right? So, now, you will, what will happen, you will increase your space and then you ask again someone, when you read the signal where it is and he again points in that direction what will happen, move even fast.

So, what is happening here, if a lot of people are pointing you in the same direction, you better start taking larger and larger steps in that direction? Does that make sense, that is how we find directions and move around. So, just like a ball gains momentum as it goes down a slope right, it is constantly moving in that direction, so it starts moving faster. So, now, can you tell me a way of incorporating this? I have been moving in a certain direction these directions are nothing, but the gradients. And now at this point someone asked me again to move in the same direction, what should I do?

Student: Take a bigger step.

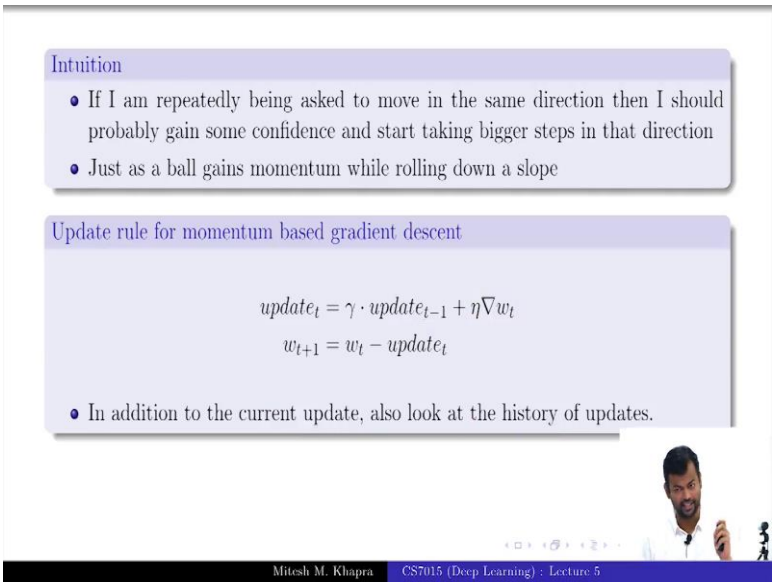
Take a bigger step. So, can you think or try to imagine, how would you do this mathematically?

Student: (Refer Time: 03:10)

So, it is probably there are a few ways to do it. So, let us see. So, what I am doing here is, this is my current gradient right? So, I asked that guy at the signal, he asked me to move in that direction. So, that is this direction and this is all my history, whatever I did till step t minus 1 ok. So, now, what I will do is, I will. So, earlier I was moving like this. This is what my update rule was wt plus 1 is equal to wt minus in the direction of the gradient right, I will moving in the direction opposite to the gradient.

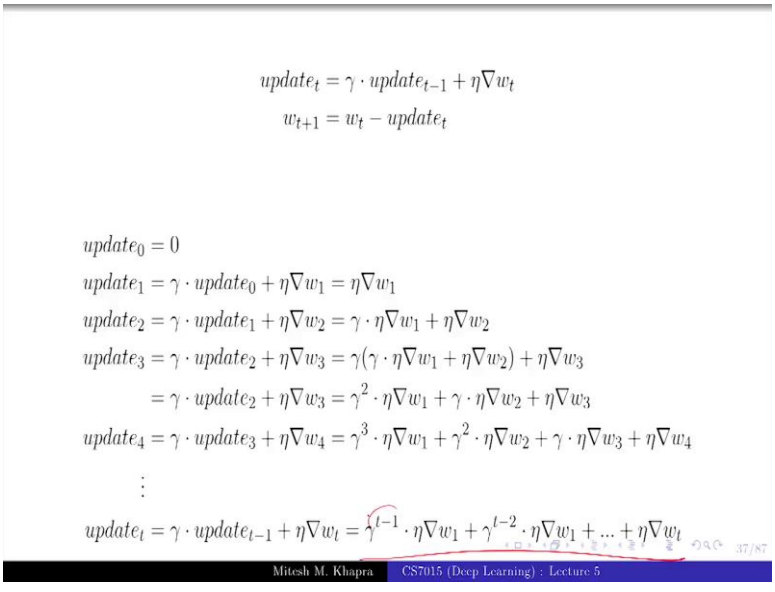
Now, what I have is in addition to that I have this gamma update t minus 1. So; that means, whatever I had done up till step t minus 1 I will also take that into account. So, I will end up taking a larger step. Is that clear? If it is not clear it will become clear on the next slide.

(Refer Slide Time: 04:18)



So, let us see what this means right? So, it basically means that in addition to the current step also look at the history. There are three guys who earlier pointed you in the same direction. So, maybe this direction makes sense right? So, start accumulating that and move faster.

(Refer Slide Time: 04:31)



So, let us just break this down and see right? So, this is what the update rule is. Sorry this is all my updates and this is the update rule. So, at time step zero my update is zero, because not started yet. At time step 1 this is what it will look like right? And this is

nothing, but just move in the direction of the opposite to the gradient, because this minus sign will come later on right in the next equation.

Now, what will happen update two? So, its gamma times update 1 plus the gradient at the current step. So, remember here everything is positive, I am adding the gradients, because my final negative sign is going to come in the next equation ok. So, do not get confused with that. Eventually I am going to move in the direction opposite that opposite will come from this negative sign.

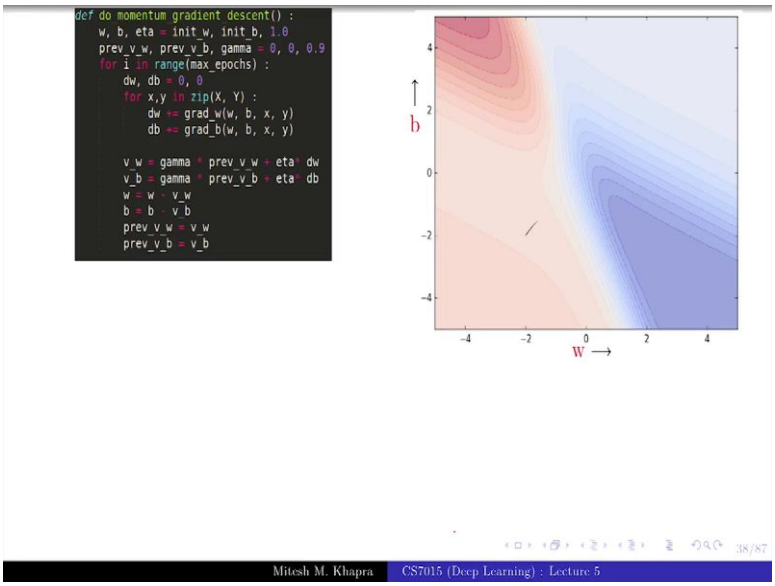
So, what is happening I am moving in the current direction plus a fraction of the direction which was pointed earlier right ok. Then does this make sense. So, can you tell me in general what is happening here at the t'th time step what is happening, what kind of average am I taking, weighted average, but it is a dash weighted average. This is an exponentially weighted average ok. So, let us look at this right?

So, when I am at step 4, I have most faith in the current gradient right and this gamma is always I will just set it to less something less than 1 right? So, I have a fractional trust in the previous gradient, even smaller trust in the previous guy and even smaller trusts in the previous guys. So, I am taking an average of all my gradients, but it is an exponentially weighted average. Does that make sense? My maximum faith lies in the current guy and then decaying faith in the previous guys.

And as I move further and further away from the last guy that I checked right I will give lesser and lesser weightage to that. So, everyone understands what is happening here. Anyone who has a problem is, just raise your hands if you understand this good.

So, in general this is going to be the formula and you see that as, as I form problem here no. As t is larger this fraction is going to become smaller and smaller right? So, you are first the first step that you take, will have lesser and lesser weightage as t increases. Everyone gets this fine.

(Refer Slide Time: 06:57)



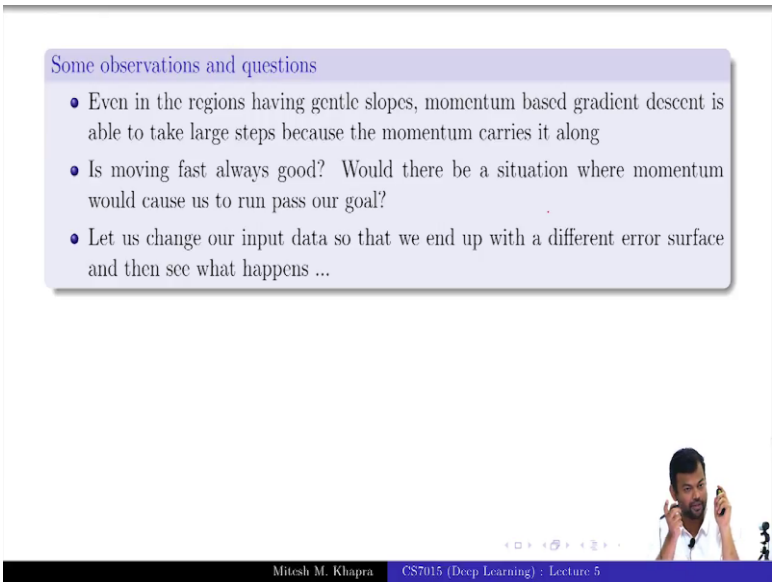
So, now this is the code for momentum based gradient descent. I will just give you a minute to stare at the code and see if it makes sense. So, this much part is ok, you are just computing the gradients with respect to all the points right? And now we are keeping this running sum ok, which is the previous gradients and the current gradient right and then you are just subtracting that running sum.

Now, this looking black curve that you see here; that is gradient this, this guy ok, this black curve that you see here, that is gradient descent when I have run it for around 100 iterations. Now I am going to run momentum base gradient descent and each click is going to be one step ok and I want you to observe what happens ok. So, slowly a red curve will start appearing on the figure.

Initially it will not be visible, so do not worry there is nothing wrong with your eyesight; one, how many if you already see the red part, I see it. 2, 3, 4, 5, 6, no now you can see it as is nothing great about7 8 9. I want you to observe something here 11, 12, 13, 14 came back right? So, gradient descent I ran it 400 iterations, it was just stuck here right this was a point and I ran this for less than like around 15 or 20 is what we counted right and so, already entered into the valley.

So, momentum base gradient descent is good, you see that wicked smile on my face and you know it is a trick question. So, we are moving fast right?

(Refer Slide Time: 09:02)



Even in the regions where the slope was gentle right that is the beginning of the, beginning of our trajectory right, this was the gentle region, even that I was very quickly able to navigate right, within 5 to 6 steps I was away from that part right? So, even in the regions where the slope was gentle I was able to move fast, but is moving fast always good.

So, would there will be a situation where momentum would cause us to run fast ago. Same thing now instead of walking you are in a car, you ask the person at the security whether I should go there, he says yes go in the right direction you keep moving there, someone else you keep accelerating, what will happen eventually? You will go fast phoenix market city then what will you do.

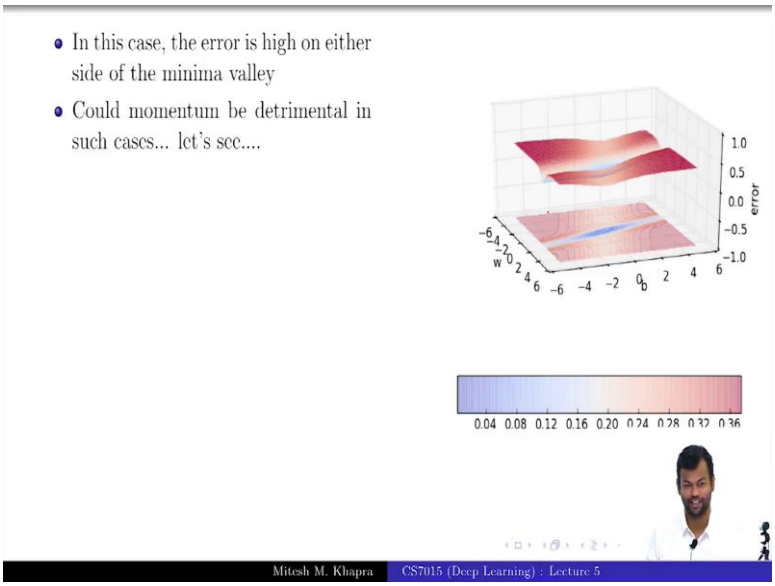
Student: Take a.

Take a u turn come back. Again while taking a u turn what will you do?

Student: (Refer Time: 09:57).

Overshoot and come to the signal and then go back again right? So, you see this you will end up taking a lot of u turns. So, let us change the input data a bit and see what happens to momentum based gradient descent.

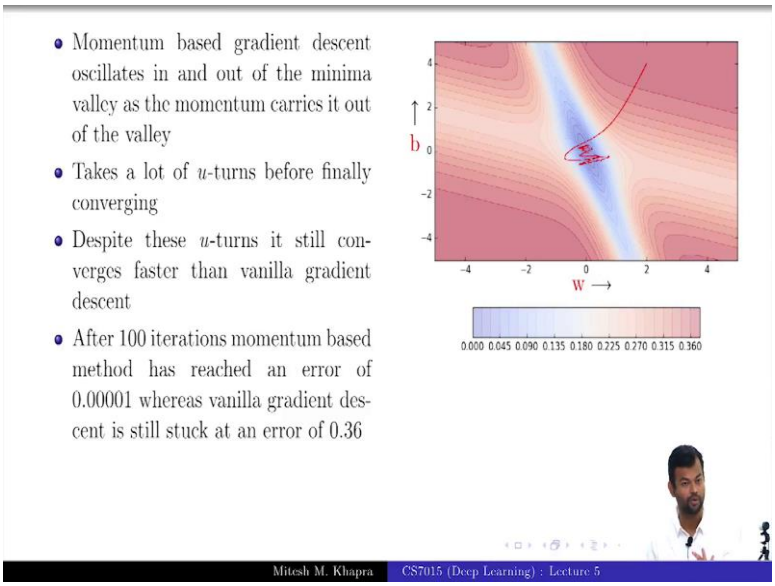
(Refer Slide Time: 10:11)



So, this is what my data looks like now. So, this is not what my data looks like, this is what my error surface looks like. So, earlier we had this error surface something like a flying carpet. Now I have a very peculiar error surface. This is again for the two parameter problem right w comma b; that means, I want to learn a sigmoid function, where I have these two plateaus at the top, the dark red regions that you see and then a very sharp valley. Can you tell me how I would have come up with this kind of an error surface, what are the points that I would have chosen. Just hold on to that part.

So, I have this kind of an error surface fine, the error is high on either side of the valley. Now could momentum be detrimental in this case, yes, no maybe I do not care, I do not care fine.

(Refer Slide Time: 11:07)



So, let us see this is the, is this the 2 d equivalent of that 3 d surface, everyone gets it. I can perfectly verify that you get it, everyone gets it I will assume right? So, these are the very high plateaus where the error is very high, very sharp and narrow valley where the error is low.

So, now again this sorry looking black curve is what I have done with gradient descent after some 100 iterations or something. Now I am going to run momentum based gradient descent and you have to help me understanding what is going to happen. Again you will soon start seeing that red curve appear 1 2 3 4 5 6, what will happen now? It is already fast that is known, it was that black curve was after 100 iterations or. So, it is fast now tell me what will happen.

Student: (Refer Time: 12:00).

He will go out, is actually almost come out of the valley right, it is almost at the top of the valley. Now what will do, take a u turn. Now what will I do. Again take a u turn. Now I will keep doing this, I will take now smaller and smaller u turns and it will converge right? So, what happens here is, because of this speedy movement and which is very analogous to that car movement which I described.

This overshoot your goal you will have to take the u turn come back if you are again careless you will have to keep taking these u turns, but you will finally, end up at the

location that you want right. It takes a lot of u turns before converging. Despite these u turns it still converges faster than gradient descent right, because gradient descent can just not move at those gentle slopes right it just cannot move from there, because the gradient is almost 0, because the slope is flat right and it just cannot move, but even with this lot of u turn and lot of rework, after 100 iterations momentum base gradient descent has reached an error of almost 0 whereas, gradient descent is still stuck at the plateau at an error of 0.36 ye. So, see you have reached the minima now.

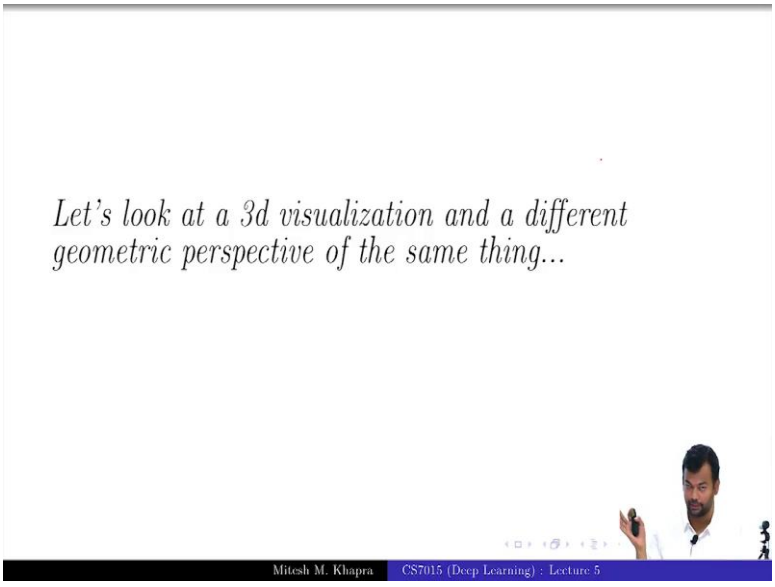
Student: Ye

Now you will be navigating there right, but you know that now your loss is very slow low. So, you could end that right, you know that your loss is very close to 0. So, you could have a condition that once you have reached something very close to 0 you could end that, even if you are making these very small movements now you could just stop there.

Student: But in the plateau regions is also 0.

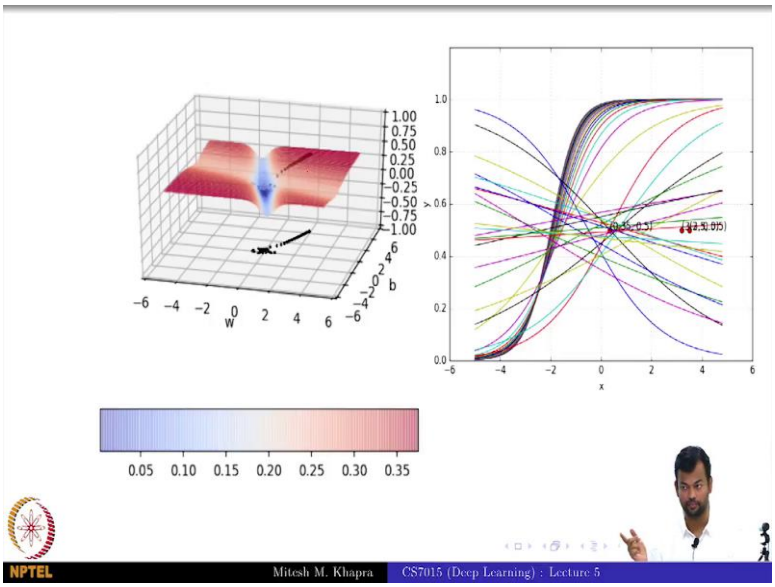
But the loss is high right? So, if the loss is high and you are not moving you cannot stop, but if the loss is low and you are not making movements you can just stop there right? So, you can just end, you can define that as your convergence condition.

(Refer Slide Time: 13:44)



So, let us look at, we will come back to 3 d now. We look at a 3 d visualization and a very different interpretation of what is happening. I really want you to understand what exactly is happening in this example which I had picked up right?

(Refer Slide Time: 13:59)



So, this is what the 3 d surface looks like view from a different angle, you have these two plateaus and the very sharp valley. Now, this is the corresponding sigmoid function where I started with. So, what I am trying to tell you is that, this is a sigmoid function corresponding to w equal to 6, oh no sorry w equal to 2 and b equal to 6.

This is the sigmoid function that I got once I plug that value. So, sigmoid is 1 over 1 plus e raised to minus w x plus b and I have plugged in the values of w and b and plotted it for all the values of x and this is the sigmoid that I got ok. So, that is my starting point. Is this good, how do you define good or bad

Student: (Refer Time: 14:43).

What do you expect at the end of training? It should pass through all your training points and these are my training points ok. Is it passing through them, no its way off right? Ok. So, now, let us start this momentum based gradient descent and what just see how my sigmoid function changes. So, right now I am on the gentle slope, even that momentum base gradient descent it is going to be fast, but not dramatically fast, because still building up the momentum.

So, it is you see that these sigmoid that I am drawing here; they are almost indistinguishable from each other. I have already drawn three sigmoids here. So, I will just go back. So, there was this initial guy then I draw drew a red one then one more and then one more, but they are all very close to each other

Now, keep viewing both these sides in parallel, what happens here on this figure and what happens to this sigmoid ok and I will ask you questions. So, still I am moving a bit slowly, because I am still building the momentum right, it takes time to build that moment. Now I have slowly started building the momentum my sigmoids have started moving towards where they should be. Everyone gets this what is happening here ok.

Now, tell me what will happen. As I enter the valley, I am almost entering the valley what will happen. I have gained this momentum now. So, my w comma b values are going to change much faster now. So, what will happen to these sigmoids, they no longer stick to each other; we will start seeing a difference they are already moving away from each other ok. So, that is what is happening to the function ok. Now you see even faster changes ok, now what will happen. I have entered the valley; this is how my sigmoid looks at this point. Now tell me what will happen?

Student: (Refer Time: 16:34).

It will go fast, what will happen to your sigmoid, how many of you know what will happen to the sigmoid ok. I will tell you what happens and then it will be obvious right? So, now, I am entering the valley, all of us know that I am going to come out of the value of the other side right? So, let us see what happens when I come out of the valley from the other side, the sigmoid changes that is why you have this situation that your error is high on both sides right, because on this side you have these kind of sigmoids, on the other side you have the other sigmoids and somewhere in between lies the solution. Where does the solution lie, at a very flat sigmoid right?

So, now I start, this is where the oscillations will happen. So, notice what will happen to the sigmoids, they will toggle between these two orientations ok, just see what happens to the sigmoids, you see it again moves keeps moving, keeps moving, it keeps oscillating around the solution and then finally, you reach the solution. So, you see that, should I repeat this.

So, when I am on one side of this valley, I have one kind of sigmoids right? Now when I move to the other side of the valley I have this others kind of sigma and take a u turn. So, when I u turn take a u turn I again overshoot and go to the other side, and this keeps happening and I keep toggling till I reach my final solution.

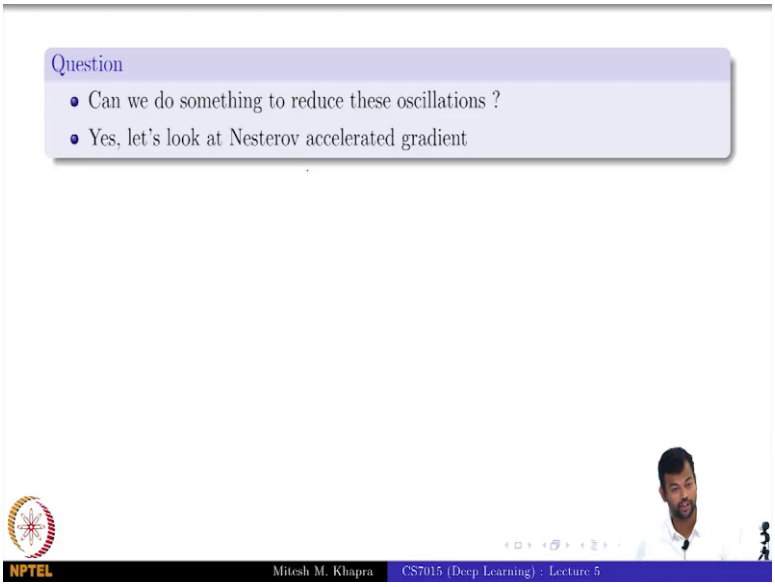
So, these are all the oscillations that you are seeing. So, can you visualize this, what is happening? Do you understand all these, relates to the actual function that you are trying to learn. So, that is why we will end this module, this was on momentum base gradient descent. Now we will see a Nesterov Accelerated Gradient Descent.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 5.5
Lecture – 05
Nesterov Accelerated Gradient Descent

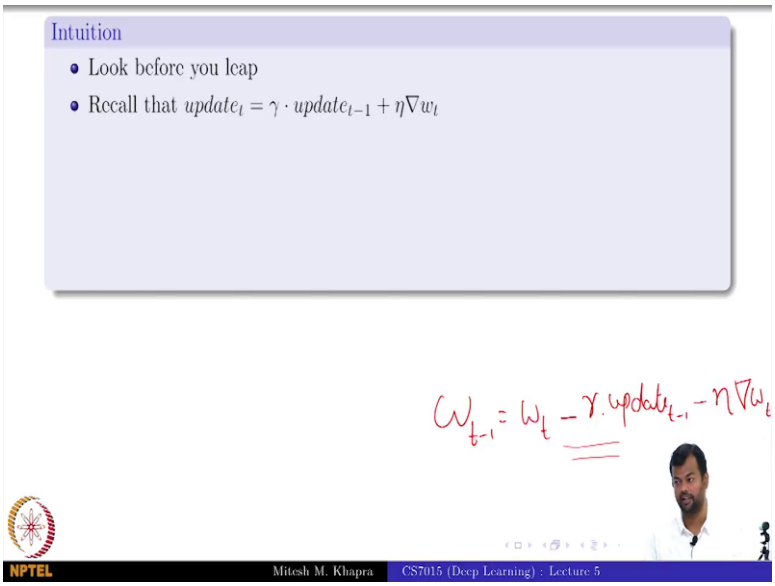
Let us look at Nesterov accelerated gradient descent.

(Refer Slide Time: 00:16)



So, now we know that momentum based gradient descent is good at these gentle regions, it moves really fast, but we do not, still do not like it, because it has this problem of oscillations. It has this problem that it overshoots its objective, its goal and then it has to take a lot of u turns. So, can we do something about reducing this oscillation? So, the answer is always yes. So, let us look at Nesterov accelerated gradient descent.

(Refer Slide Time: 00:41)

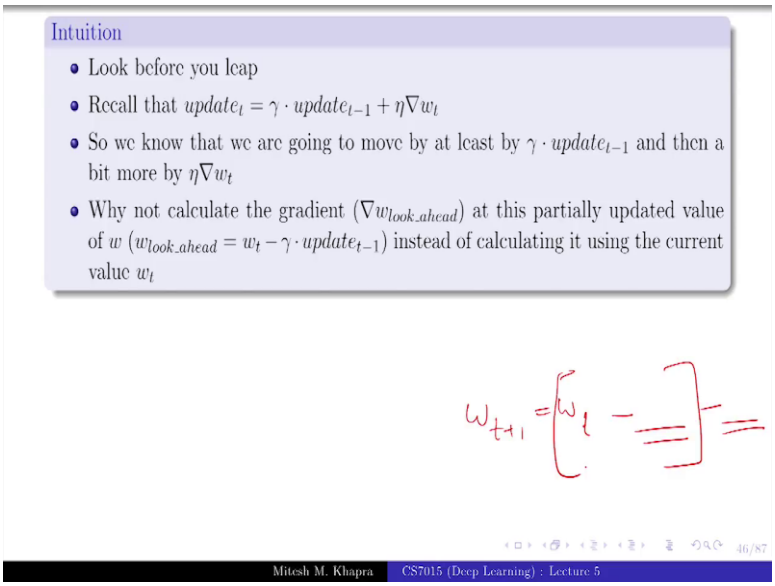


So, the idea here is very simple, look before you leap ok. Now remember that this was the update rule for momentum based gradient descent ok and I will write it down again wt plus 1 is equal to wt minus gamma into update t minus 1 minus eta into the gradient at the current point.

So, you see that actually I am taking two steps; one is this step and then one more step and I could just this is one way of visualizing right, that I move according to the history and then I move a bit more according to the current gradient. So, everyone sees that there is a two step movement happening here.

Now, can you think what could have been done, look before you leap. So, we will see what we can do.

(Refer Slide Time: 01:34)

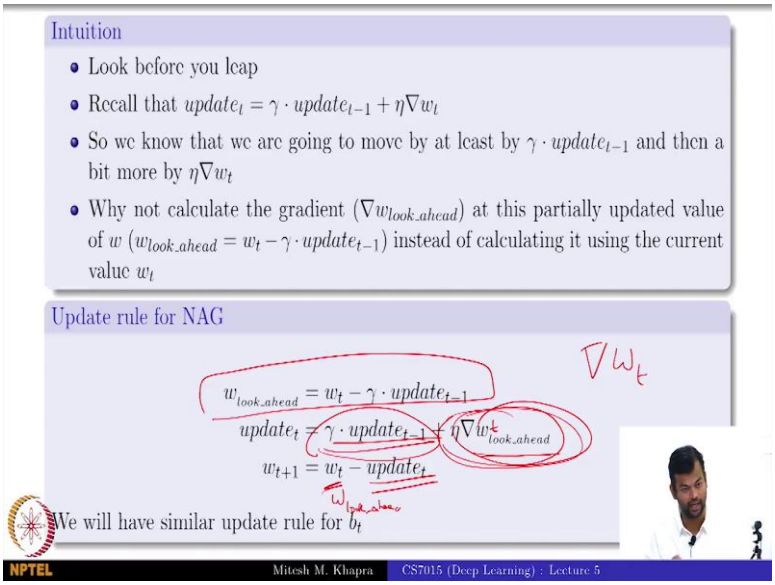


So, we know that we are going to move at least by this one and that is fixed we know that our history is telling us to move at least by this one and then we will move a bit more by the gradient.

So, now can you think about it, I am at least going to move this much. What if I had some way of looking ahead and then do something at that point, this is what you are saying. Of course, I can verify it, but I am sure it will become clear once I show the equations, but I just want you to think about it a bit wait, it is very simple it will become absolutely clear once I show you the answer, but just think about it a bit.

So, here is the answer it, why not compute the gradients add this look ahead point right. So, you are again adding it in two steps; minus the history and then minus the current gradient. So, take this value, call it the look ahead point. I know that I am going to move by this much. So, let me not compute the gradients at the current point, let me move by this much, then compute the gradients and see what happens at that point.

(Refer Slide Time: 02:52)



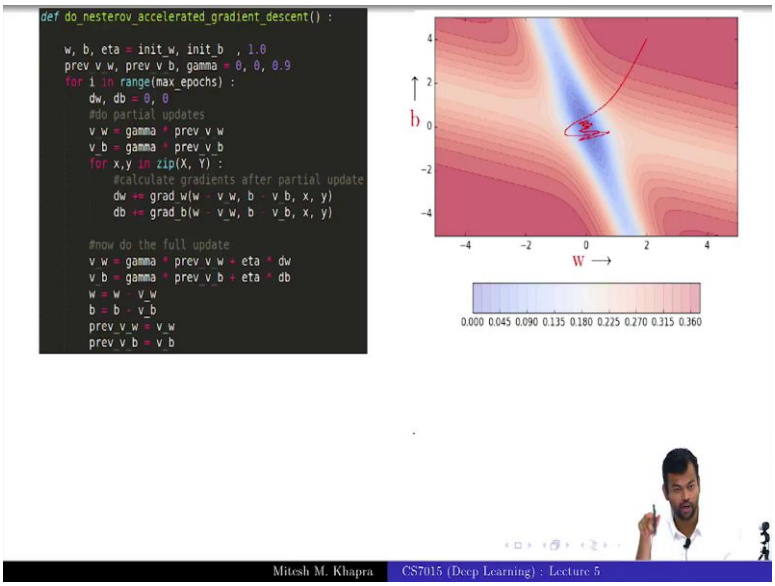
So, this is the equation right that first I move by that one step. I had to make a two step movement. So, I will move by that one step right, then I will compute the gradient at that position, not at my current position right, this was earlier gradient at point t, now I have already moved a bit. So, I can compute the gradient there and then move in the direction of that gradient.

So, you understood this that there is a two step movement right wt minus history minus the current gradient, gradient computed at time step t ok. Now you know that you are already going to move by the history right? So, why not just move there and then compute a gradient at that point you are anyways made some movement you compute the gradient at that point and then decide which is the direction to move in right?

So, that is what this look ahead value is. I know it is still not clear to many of you and I am very confident it will become clear in the next 5 minutes; we will show you one more visualization for this, but this stay with, stay with me for a while. As long as you get the intuition I am fine I will move ahead and then I will explain it again in a different way, this is fine ah, that should become clear good that you asked that question ok. So, ask me again on the blank slide that I have and then I, it should be complete.

So, for right now let me just show you what will happen with the code, and then I will again explain it with a different way.

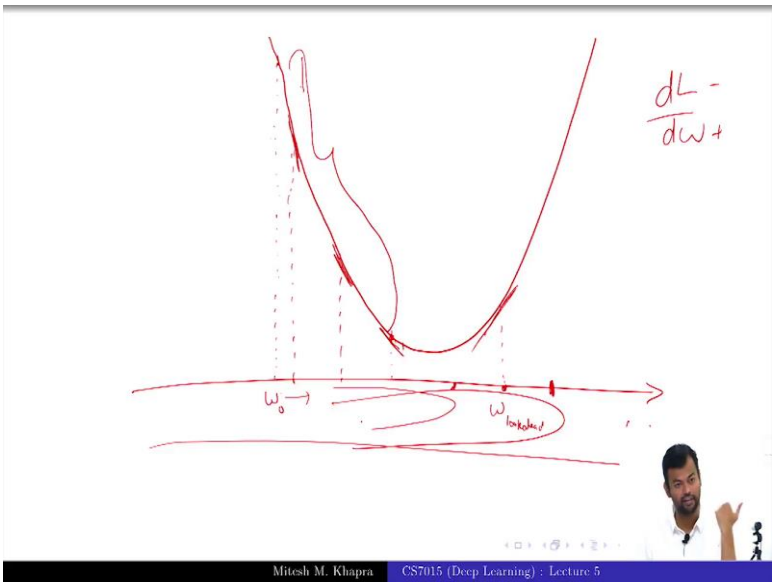
(Refer Slide Time: 04:35)



So, this is what momentum based gradient descent it ok. Now let us see what nested or accelerated gradient descent will do. Again the code is simple you can just read it up and I have started executing, you see this blue curve coming over there fine, ok and now I keep running this. Now what will happen? You see that all the u turns of the blue curve are inside the u turns of the red curve.

So, the objective is being achieved at least empirically I have showed you that right, its taking shorter u turns. What is probably not clear to all of you is, why is this happening. Is it clear to everyone, why is this happening, can everyone visualize that ok. So, let us see why this is happening, I will give you an alternate explanation for this.

(Refer Slide Time: 05:38)



So, suppose this is my error surface right on a two by and I have a single variable with respect to which I am trying to optimize. So, this is my w. I started off with some initial value w naught.

Now, what is the gradient at this point; positive negative, negative right, because when I am going to increase w the function is actually going to decrease right. So, right. So, the slope is negative. So, where will I move, this is the number line right. So, this line is actually the number line, because it is a single variable. So, where will I move positive side of the number line or the negative side of the number; positive side. The derivative is negative; I am going to move in the direction opposite to the derivative. So, I am going to move in the positive direction right. So, I will end up somewhere here. Is that clear, fine with everyone.

So, now I am somewhere here, what is the derivative at this point. Now what is the derivative here; positive negative, negative right when I am increasing w my loss function is decreasing. So, my dl by dw is going to be negative, this is positive this is negative. So, again I will move in this direction.

So, what is happening a lot of negative updates are getting? Sorry a lot of positive updates are getting accumulated. And now because of my momentum I am not going to move only by this derivative I am also going to move by the history right. So, I will end up somewhere further.

So, now at this point what is the derivative, again negative, when I am increasing w the function is decreasing. So, what is my update positive or negative? Positive. So, now, you see that a lot of positive updates are getting accumulated right, my momentum is building up. So, now, what will happen? Now if I just move further, then again I will get a let me just put it here right. So, I am again moving largely in the positive direction, because this guy is also positive, all my history was also positive. So, I have moved in the positive direction

Now, what will happen at this point, what is the derivative here. No it is still its negative sorry. So, again I am going to move in the positive side of the number line ok. Now at this point I want you break down the movement into two points; one is what my history was telling me, which was all these positive updates, but of course, I will not make such a large update, because I am waiting them exponentially right so, but its telling me to move in the positive direction ok, and I know that the gradient at this point is negative, but I want you to ignore that for now. I just want you to focus on the history. If I just move according to the history where will I end up? I will end up somewhere here right, because the history is very positive. So, I will keep moving in the positive direction and this is my w look ahead.

Now, what will happen if I compute the gradient here?

Student: Positive.

The gradient is.

Student: Positive.

Positive. So, where will I move?

Student: Negative.

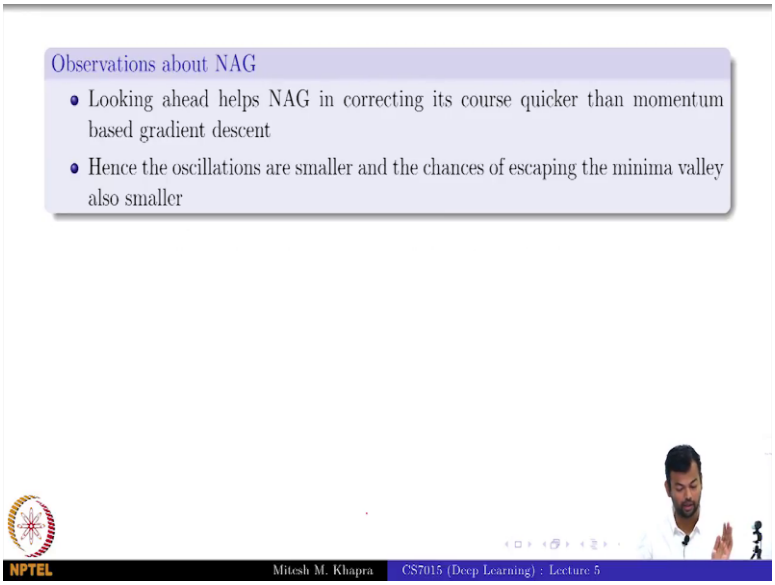
Negative. So, you see now why momentum works, because you are able to look ahead to this point. Instead of what should I have actually done is I should have looked at the gradient at this point, the history is positive; the gradient is also telling me to move positive. So, I would have moved a large positive and I would have ended somewhere here. Instead I just moved by the history, I checked where I end up I end up here.

Now, let me see whether what is the gradient at this point? Have I already overshoot my overshot my objective, when would I overshot my objective, it has the sign of the gradient changes right, it became from negative to positive and now since its positive because as I am increasing w, the loss is also increasing. So, now, where will I move? Negative.

So, now, what is the second step actually its again bringing me close to here. So, instead of taking this large u turn, I end up taking this small u turn. Is this clear to everyone now? How many if you still do not get it, how many if you get it now, good sure ok. So, this is what and now we can relate it to what was happening on the figure

So, let us go back right. So, you saw that I was making these smaller u turns, because when I was at this point right, I already moved by the history I knew I would land up somewhere here, where I would need to go back right. So, I already accounted for that and made a very small movement. Is this clear, everyone gets this how the Nesterov of accelerated gradient descent works, sure raise your hands.

(Refer Slide Time: 10:42).



So, looking ahead helps nag in correcting its course quicker than momentum based gradient descent right? So, it is already looking ahead where do I land up and already making a correction if required, if not required it will again move in the right direction right. So, the update is this guy plus the gradient and my update happens on the original value not on the look ahead value.

So, her confusion was perhaps that I am doing w look ahead minus update, where this update again has this quantity you know that is what your confusion was, but I am not doing w look ahead I am using wt there; everyone gets this. So, that is where ah. Now it is clear that why the oscillations are smaller in the case of nag and it is able to correcting its course quicker.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 5.6
Lecture – 05
Stochastic and Mini-Batch Gradient Descent

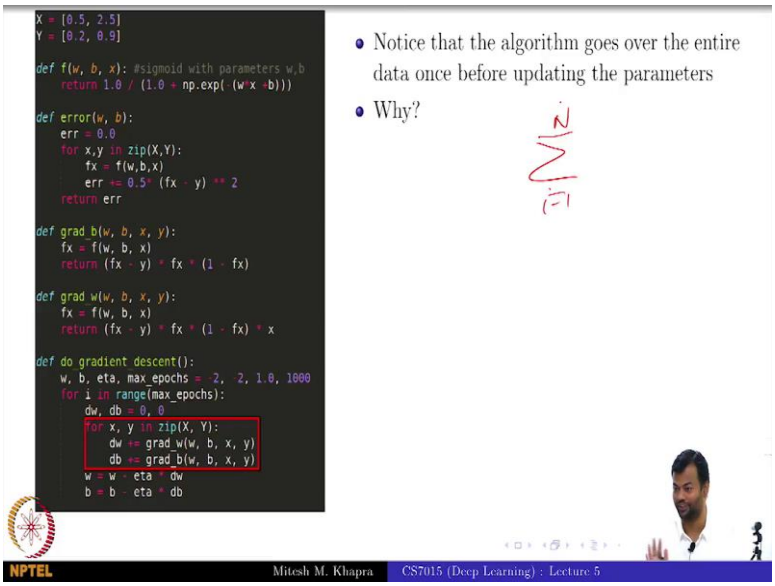
Now, we look at Stochastic and Mini Batch versions of these algorithms.

(Refer Slide Time: 00:20)



So, we will digress a bit, actually we should have ended up somewhere else, but I was just going to digress a bit.

(Refer Slide Time: 00:25)



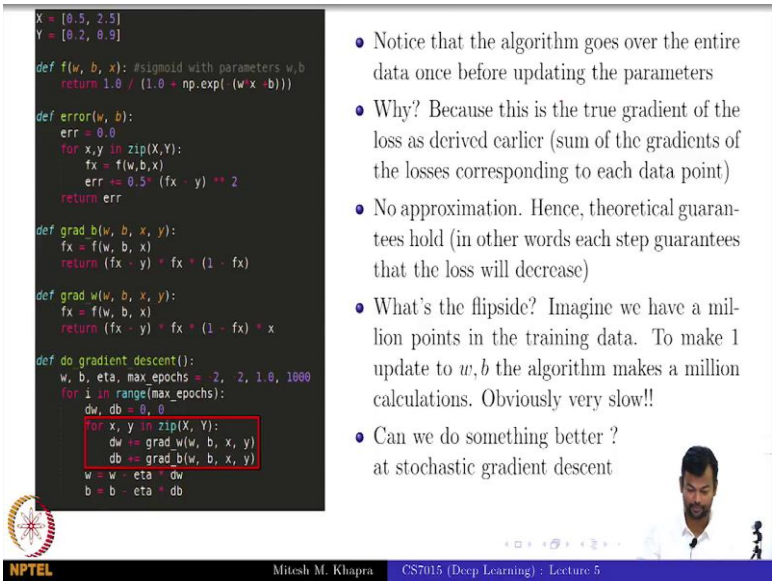
So, this is the original gradient descent code that we had, and I have highlighted something in this red box.

So, notice that the algorithm actually goes over the entire data once, before making an update. It has going over this entire for loop, which is over all the data points. Of course, in this toy example I had only two data points, but in I practice I will have many many data points. I go over all the data points compute the derivatives and then make this one update.

Student: (Refer Time: 00:57).

Because that is the right thing to do ok. This was the exact formula that we painfully derived right that the gradient with respect to the loss function right, which we had the summation i equal to 1 to n remember, and the true derivative was a sum of the derivatives with respect to all the data points; that is what we analytically derived and hence we are doing that, it was that is the right thing to do, not for any other purpose ok that is what it should always be right. So, that is the right thing to do, because this is a true gradient and we actually derived it.

(Refer Slide Time: 01:28)

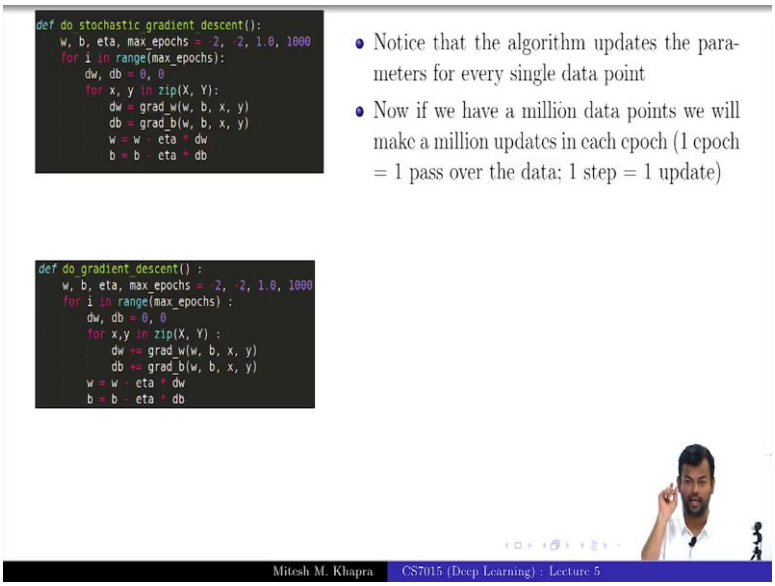


And hence this was not an approximation, so all the theoretical guarantees hold. If I do this I know that now this is the true gradient or the true derivative and if I move in the direction opposite to the gradient everything falls in place, because I proved it using Taylor series

But what is the flip side of this. This is the right thing to do, but what is the flip side. If you have millions of point, we will go over all these million points and make this one update. Now imagine the consequence, when you are in a plateau region right, even that momentum or whatever your movement in the plateau is going to be relatively smaller right. You are going over these million points and making that tiny delta update right. So, imagine how much time it will take your algorithm to converge, you get the problem.

So, the algorithm will take a million calculations and then make one tiny update to your w ok, this is going to be very slow. Can we do something better always right. So, let us take a look at stochastic gradient descent fine.

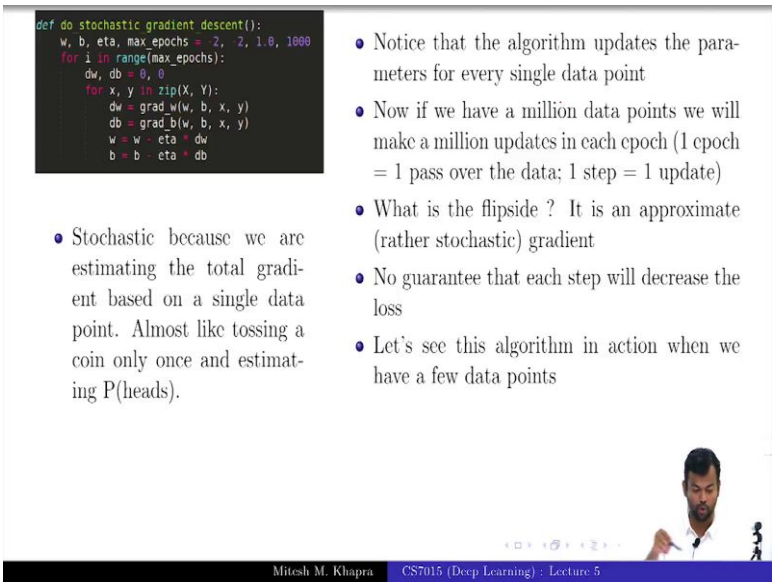
(Refer Slide Time: 02:36)



So, I have done a very subtle change to the code what is it, do not tell me indentation, but that is what I have done, so you can tell me that. So, what is happening now? For every data point I am making an update to my w values.

Now, the algorithm updates the parameters for every single data point. If you have a million data points, how many updates will be make in one pass over the data? A million for every data point will make an update right. So, that slowness factor in what is known as batch gradient descent right, batch gradient descent is when you look at the entire data and then make one update.

(Refer Slide Time: 03:19)



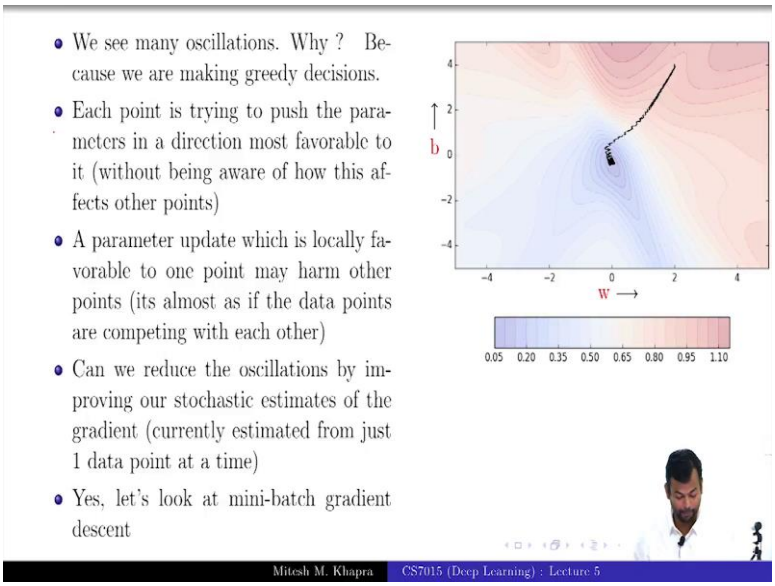
What is the flip side, what does this module titled stochastic gradient descent? So, what is the flip side? These are not the true gradients. The true gradient is summation over all the points. Now this is no longer the true grading, this is just a point estimator, this is just a approximation of the gradient right. And stochastic, because we are calculating the gradient based on a single data point right, it is a sampling one data point and computing the gradient that this is what the entire population looks like right.

This is almost similar to tossing the coin once and saying that this is what the probability of heads is. If it lands at heads then the probability is one; otherwise its 0 right, you see the error you see the problem with that right, as opposed to tossing the coin a thousand times and then deciding the probability is just tossing it once. So, this is always going to be a erroneous right this, this is going to be bad

So, now there is no guarantee that each step will decrease the loss why, because the guarantees were only when you are doing the right thing which was to compute the gradients over all the data points. Now there is no theoretical guarantees right, because it is all stochastic now. So, it is possible that in a particular data point your loss might increase also, the overall loss on the data. With respect to that point it might decrease, but the overall loss right

So, now let us see this algorithm in action and I want you to make certain observations about this. So, this is the code that I am going to run now. So, let us see.

(Refer Slide Time: 04:44)



So, I will start and you have to observe and let me know and this is really becoming an eye test for all of you, but that is good. So, for nothing interesting to observe or already maybe.

Remember I am running gradient descent, this is not momentum, not Nesterov this is gradient descent ok, I have already given you the answers. What do you observe?

Student:

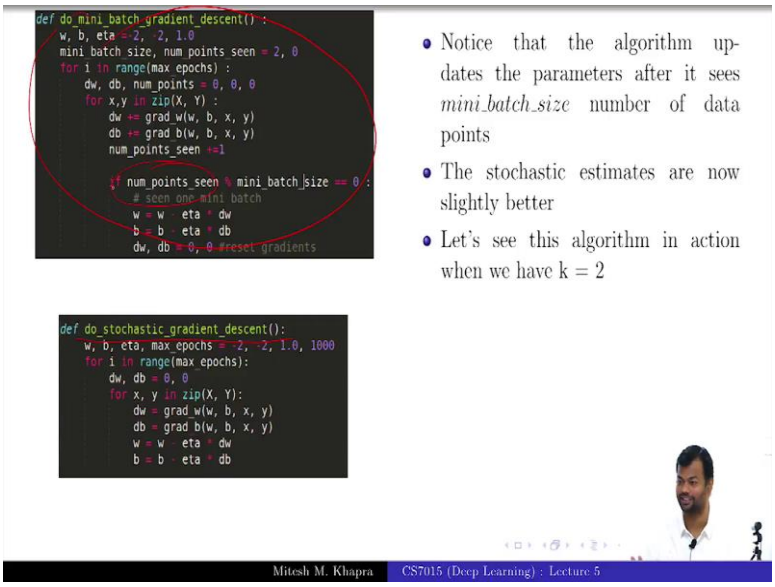
I can still pretend an answer a let us do that. We see many oscillations, why? Why do we see the oscillations? Are these oscillations the same as the oscillations that we see in momentum? No these are different, everyone gets that right, why are there oscillations. What is each click here correspond to one data point right. So, what is happening here? Because we are making greedy decisions right we are looking at one point, this point says to decrease the loss with respect to me move in this direction and we blindly move in that direction.

Now, we look at the next point, it says oh no no wait you need to move in this direction. So, we again move in that direction. So, all these points are actually trying to just make things better for themselves. They are not thinking about what is happening to all the other points in my data right. So, all these points are actually competing with each other.

So, some decision which I took with respect to where to move, which was locally favourable for one of these points may not be good for the other point right. Hence I keep these tiny oscillations which I make, these are the stochastic noise that you are seeing now.

Now, can we reduce the oscillations by improving the stochastic estimates? Always yes fine. So, let us see what do I mean by that.

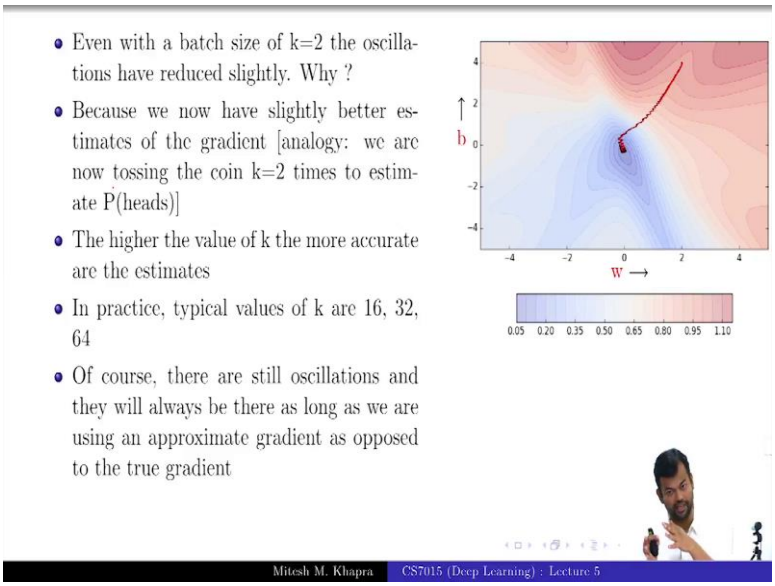
(Refer Slide Time: 06:57)



So, we look at a mini batch version of this. So, what I am going to do is, instead of. So, this code is actually for mini batch stochastic gradient descent, it is a very minor alteration on the stochastic gradient descent, I will just let you stare at it for a minute or. So, what I am doing here is, I am, instead of doing it for every point, I am waiting for a certain number of points and then making the update right, that is what I am doing here.

Now, for this I have kept k equal to 2 what does that mean? I look at two points compute the derivatives with respect to them and then make an update for two points at a time, what do you expect? No what do you expect with respect to this code.

(Refer Slide Time: 07:47)



So, let us see we will try to run this now and you will start seeing a red curve here and make some observations about this. So, this is the red curve, yeah its visible. Oops I do not read any of those.

Student: (Refer Time: 08:07).

If you need to fix this right, these bullet us should come only after the curve has finished this journey ok. Do not read any of that ah. So, what do you see about the red curve? It is completely contained inside the black curve; that means, its oscillations are smaller than the black curve right. Does that make sense why this is happening, because now you are not listening to just one point, you are listening to two points and then at least you are doing something better right instead of just taking one.

So, what is the analogy with respect to our coin toss experiment? You are tossing the coin twice and then deciding what is the probability are heads or tails right. So, it is always going to be slightly better than tossing it only once right. And now what would happen in the limit if I keep increasing this, you will end up with a batch gradient descent where you look at the entire data

So, looking at only one data point is bad, because it is very noisy, looking at the entire data is bad, because it is very time consuming. So, you need to do something in between which is mini batch gradient descent ok, and typically you look at values of 16, 32, 64

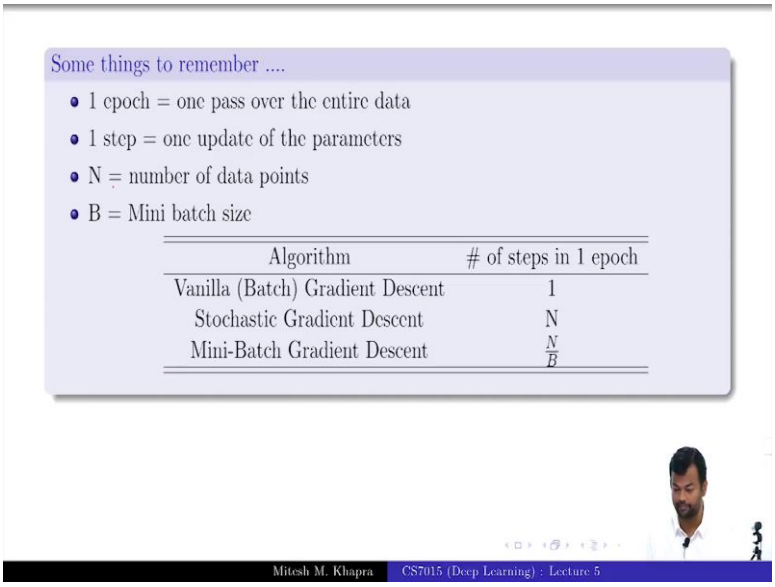
but it also depends on the amount of data you have and if you have a billion points you might actually want to look. Because if you have a billion points and you have a batch size of 64 you will take 1 billion by 64 times to finish the data once.

So, you might want to keep a larger batch size at that point right, but just ignore that, but you will try different batch sizes and see which one works better. So, in the assignment I will be asking you in to experiment with bad sizes, yes ok. No sorry wrong question, I will be asking them to implement stochastic and mini batch also or only vanilla.

Student: Mini batch.

Mini batch fine that is fine ok. So, you will see this in your assignment. So, everyone sees what was the difference between stochastic and mini batch. You have better estimates now and therefore, this red curve is contained inside the black curve fine.

(Refer Slide Time: 10:08)



So, you have some things to remember; one epoch get used to this terminology, 1 epoch is 1 pass over the entire data. One step is one update to the parameters, n is equal to the number of data points and b is equal to the mini batch size. Now you have to fill in the second column. In vanilla or the batch gradient descent what is the number of steps that you take in 1 epoch.

Student: 1.

1 in stochastic gradient descent

Student: N.

N n.

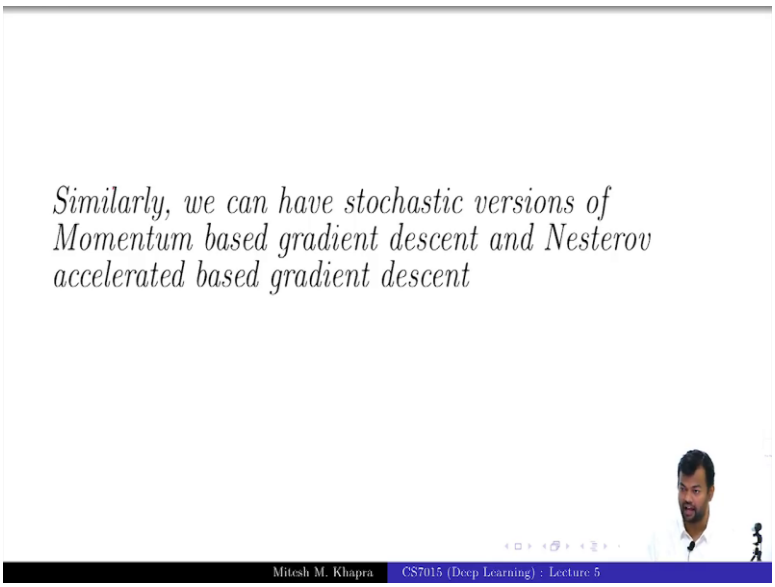
Student:N.

In mini batch gradient descent.

Student: N by b.

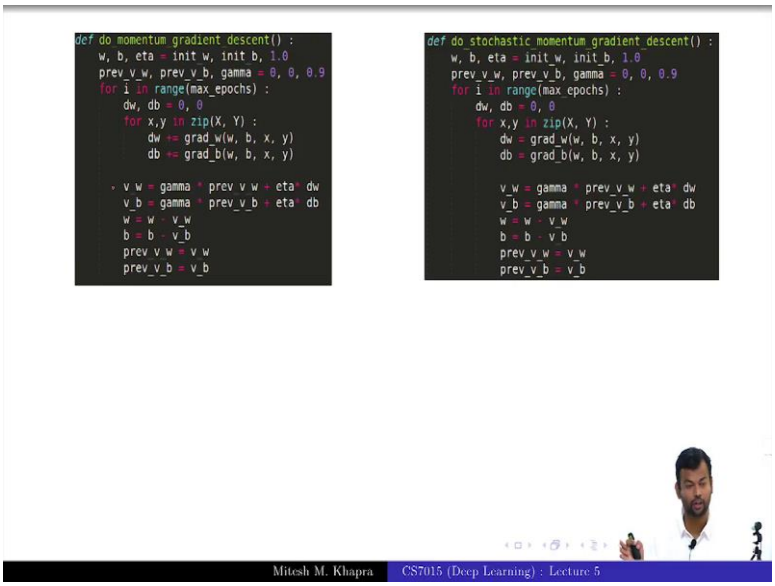
N by b, everyone gets that. So, get used to this ok. So, this epoch step batch size all this is something that you will see regularly when you are reading papers on deep learning fine.

(Refer Slide Time: 10:53)



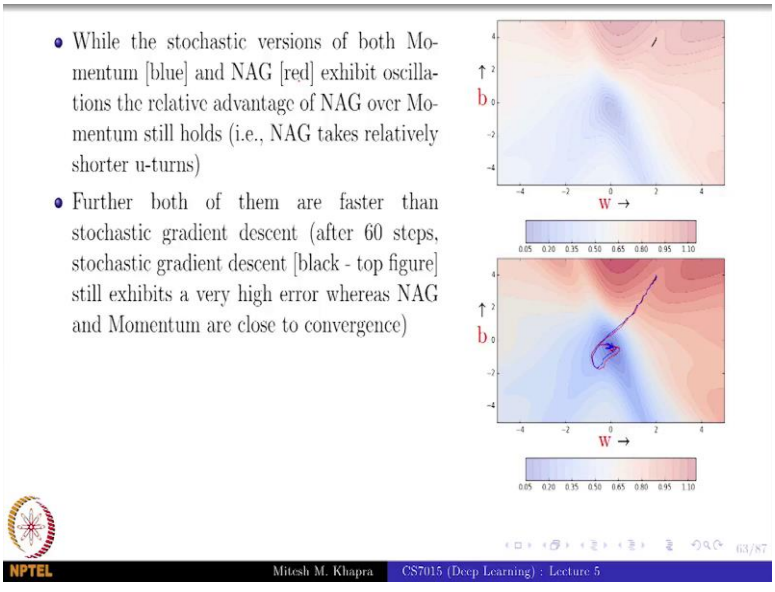
So, similarly we can have the stochastic versions of momentum based gradient descent and Nesterov accelerated gradient descent.

(Refer Slide Time: 11:02)



So, these are just the codes, it is very easy to see what is happening here. Again basically this is just an indentation right. So, if you look at the difference between the two codes, I have just indented it inside; that means, I am making these updates for every data point right and same thing you could do for Nesterov also.

(Refer Slide Time: 11:21)



Now, let us see ah. This guess what is it? This is the gradient descent, stochastic gradient descent. Now let us see if you have really understood nag and momentum based gradient

descent. One of these curves here corresponds to stochastic nag, the other one corresponds to stochastic momentum. Tell me which one is which

Student: Blue pill.

Blue pill red pill. Blue is.

Student: (Refer Time: 12:01).

How many of you say that. Ok I am confused ok. How many of you say that blue is momentum, how many if you say that red is momentum? Oh there is so many, you do not have an opinion

Student: Sir not clear.

Not clear, I will buy that. So, ok. So, look at this, who is taking longer u turns; momentum or nag? Momentum roughly which guy is taking the larger u turns?

Student: Red guy.

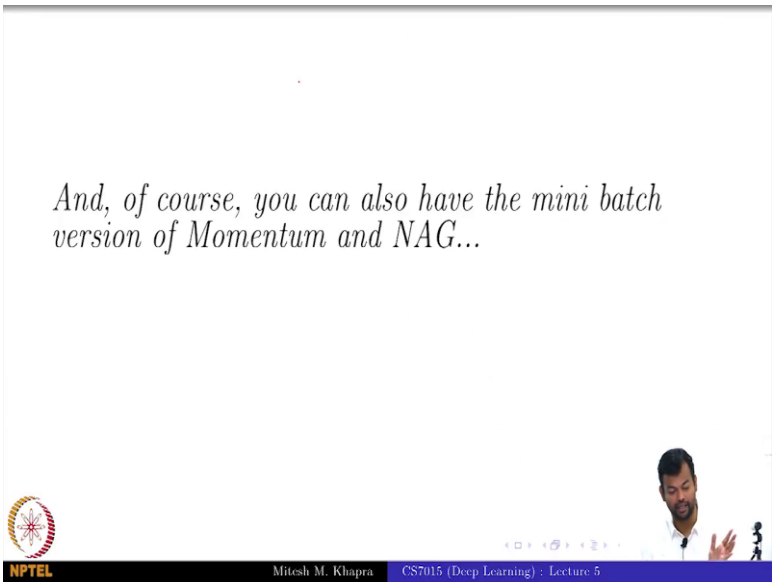
Red guy right, I mean roughly speaking. There is only one point to judge by this here, because here they are almost same and that could happen in practice right, because this is now noisy. So, the red curve corresponds to.

Student: Momentum.

Momentum, because it is taking a larger u turn, we saw that momentum takes larger u turns and the blue curve is corresponding to nag ok. So, no I remember this was an error on the slide yeah. So, this has to be red and this has to be blue, so ok. So, the momentum is actually red and the nag is blue because it is taking a shorter u turn and the reason you do not see it very clearly is because both of these are running in this stochastic mode.

But you still see the relative advantage of them that nag still takes shorter u turns, both of them are faster, still faster than vanilla gradient descent.

(Refer Slide Time: 13:35)



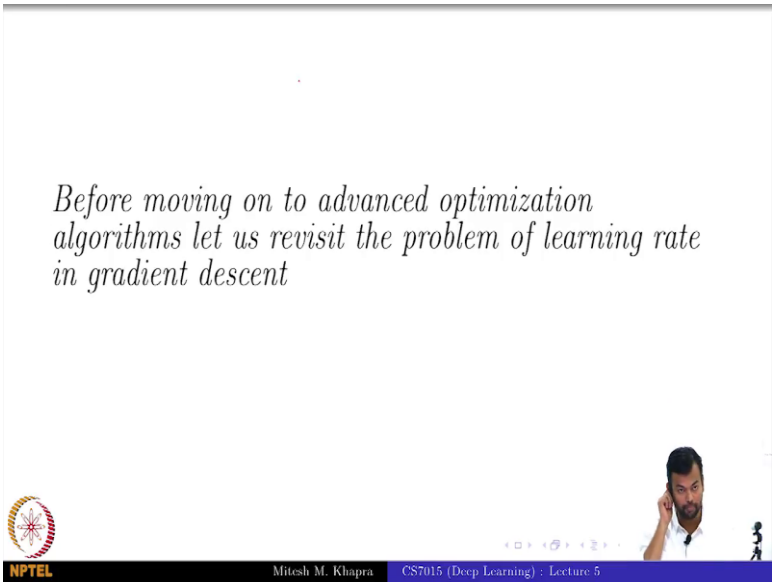
You see that black curve at the top and both of these are faster than them, both of them, all three have run for the same number of iterations. After 60 steps you see what happens to stochastic gradient descent and what happens to nag and momentum basically gradient descent. And of course, you can have the mini batch versions of momentum and nag also.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 5.7
Lecture - 05
Tips for Adjusting Learning Rate and Momentum

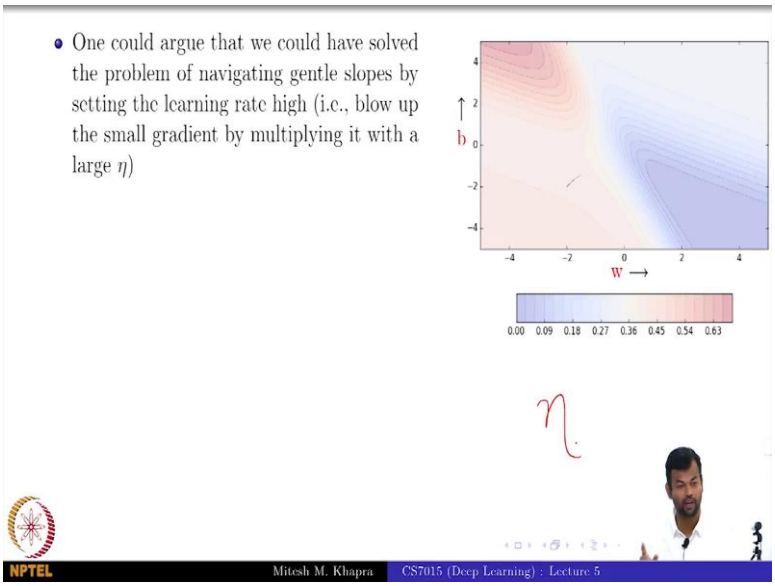
Tips for adjusting the learning rate and the momentum.

(Refer Slide Time: 00:17)



So, before moving on to these slightly advanced optimization algorithms, we will revisit the problem of learning rate in gradient descent.

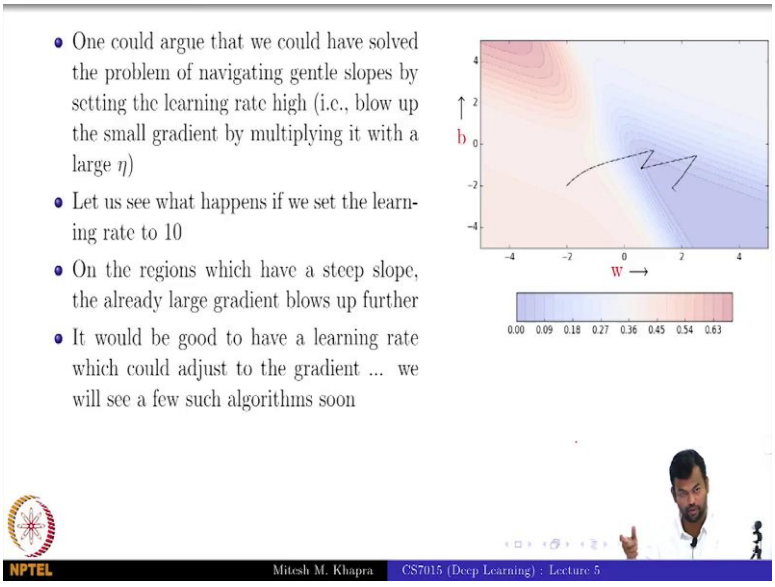
(Refer Slide Time: 00:24)



So, one could have argued that we could have solved this problem of this slow movement on the gentle slope by increasing the learning rate. Remember that we have this eta and we deliberately chose to be conservative, that we will take a small value for the eta, but what if I just blow up the eta; I could just take a very large eta, what would happen? It will overshoot right?

So, what will happen is, I will see what happens when I take eta equal to 10 ok. So, so I will see what happens, when I take eta equal to 10.

(Refer Slide Time: 00:57)

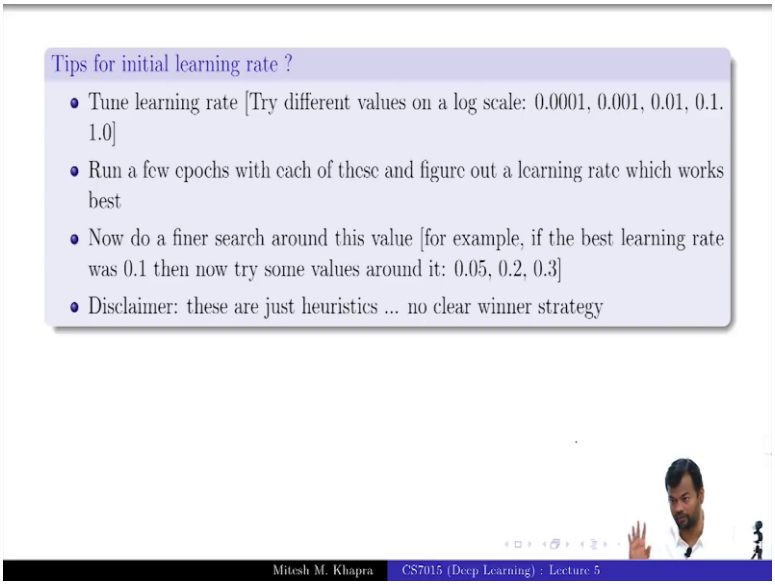


So, this is step1, step 2, step 3. Its moving very fast on the regions where the slope is gentle, but it also moves very fast, much faster on the regions where the slope was already steep.

So, when the gradient was actually high, you ended up blowing it further by multiplying it with the eta which is 10. So, it is again going to have this effect that you will move much faster in the steeper regions and again you will see these oscillations, because you will overshoot your objective. Does that make sense right? So, it is not that you can always choose a high eta and get away with it

So, what do you actually want, what is your wish list regulate theta. You want a adaptive eta right that it somehow figures out that I am on a gentle slope, so I should move slowly, I should move fast and I am now on a very fast loop, so I should move slow. So, this having this 1 eta is not working for every point on the error surface right, for everywhere on the error surface, is that clear ok so, ok. So, we will see such algorithms soon where we try to adjust this learning rate.

(Refer Slide Time: 02:06)



Now, here are some tips for the learning rate. So, how do you, if you are just going to deal with this gradient descent or nag or momentum, how do you adjust these learning rate. So, how do you fix a learning rate? So, a learning rate is typically something known as a hyper parameter. So, why is it called a hyper parameter? So, what are your parameters?

Student: Which I learned.

Which I learned using the objective function; eta is not a part of the objective function, you are not computing radians with the respective to, it is a hyper parameter. So, you will try to tune this hyper parameter. So, what you will do is, in practice you could try these different values on a log scale. Next what will you do? Run this all these for a few epochs, note down the dash, just note down the loss function.

So, run all of these with different learning rates, for say 5 epochs, you will get some loss right? Now which one will you pick? The one which led to the maximum decrease in the loss, I will keep that learning rate and now what you will do? You just stick to that I started off with a dash scale.

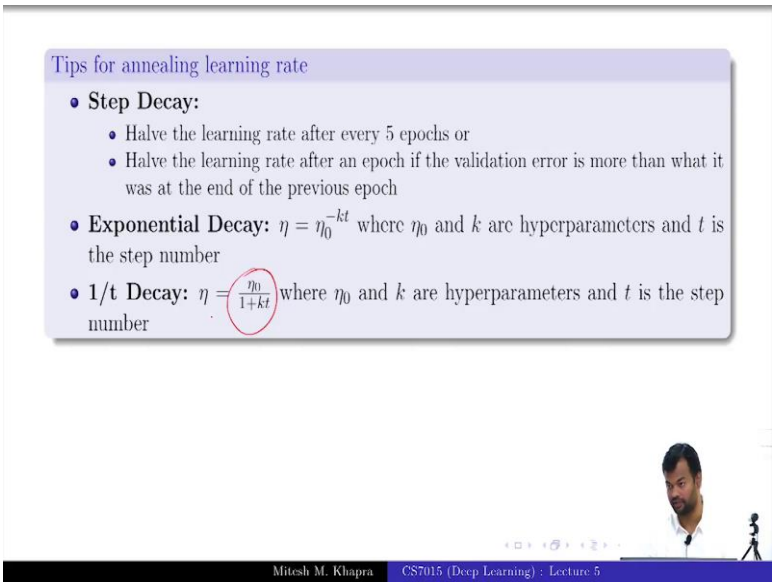
Student: Log scale.

Log scale now what will you do ok. So, now, run it for a few epochs. Figure out which of these learning rates on the log scale works well. Now do a finer search around the best learning rate that you discovered right? So, say 0.1 was the best on the log scale. So, now, look at 0.2, 0.3, 0.4, 0.5, look at values around it and see which one works better.

So, this is how you will tune the hyper parameters; otherwise there is a very wide range right, if you put tune from 0.0001 to 0.1, there are just too many values to consider. So, we will have to do this log scale and then a linear scale, will that make sense.

These are just heuristics, there is no guarantee that will always work or which of these are clear winner strategy, but you have to try this. So, tuning a learning rate is an important part, when you are working in deep learning. So, at least when you are working with gradient descent or nag or momentum based gradient descent.

(Refer Slide Time: 04:02)



Now, here some tips for annealing the learning rate. So, there is something known as step decay. So, what you can do is, halve the learning rate after every 5 epochs. Can you tell me the intuition for this? What do you expect after 5 epochs? That you have moved enough and now you are closer somewhere to the solution. So, if I closer to the solution, if I closer to phoenix market city you want to move fast or slow?

Student: Slow.

What will you do?

Student: (Refer Time: 04:32).

Decrease the learning rate right. So, after every 5, now this is again what is so sacrosanct about 5, it is just a magic memory, so this is again hyper parameter. So, you could fix some number of epoch and after these I will just halve the learning rate ok. Now this second one is what my favourite is and I typically use this, what I do is? I compute the loss after epoch t, I run epoch t plus 1, I compute the loss again. If the loss has increased what will I do? I will just throw away all the updates that I have made in this epoch; I will decrease the learning rate and again learn, again start this epoch. What do I mean by throw away all the updates?

Student: (Refer Time: 05:17).

So, after epoch t I will save my model, I will save all the w values that I have computed and I will let it run for one more epoch. After this epoch if my loss function actually increases, I reload this model which I had saved, half the learning rate and then run this epoch again. Does that make sense. So, I have run till epoch t, I have some values of w's and b's, I will save this values, I will just save it as a numpy array.

Now, I will with the same learning rate that I have been using so far I will run the epoch t plus 1 ok and I get some new values of w comma b right. I will plug this into the loss function; I will plug this into the loss function I will get two loss values. If this loss value is greater than what I was at the previous time step; that means, things did not work out well in this particular epoch.

So, I will throw away all these updates. I will just reload the model which I had saved, I will just start from where I was at epoch t, I will decrease the learning rate I will make it half and run this epoch again right and hopefully now I should do better, because there is something. I am just making a hypothesis that the reason I did not get to a better loss function, was because my learning rate was not adapting to it.

So, I will just halve the learning rate, because this solution was good, this was a low loss function. I just want to be something around it; I do not want to make any drastic steps. So, I will just half the learning rate from there. So, then you not see this drastic change that, your loss function should not improve. So, first of all local minima is known problem in deep neural networks.

So, what happens is that, in deep neural networks you do not have something which is like a neat convex function as your loss function right? It is a non convex function which means there is no one unique minima, there could be several minima and there are several analysis which show that a lot of these minima are equivalent. So, in practice these are the things that you do. Either once you reach a minima you just stay there. The second thing that you could do is, you have trained your algorithm trained your parameters for say 100 epochs and you have stopped now

Now, again go back and start with a different initialization, you started with some w naught b naught and you have reached to some solution keep this solution. Now start with a different initialization; that means, if you look at your wb plane, you have started

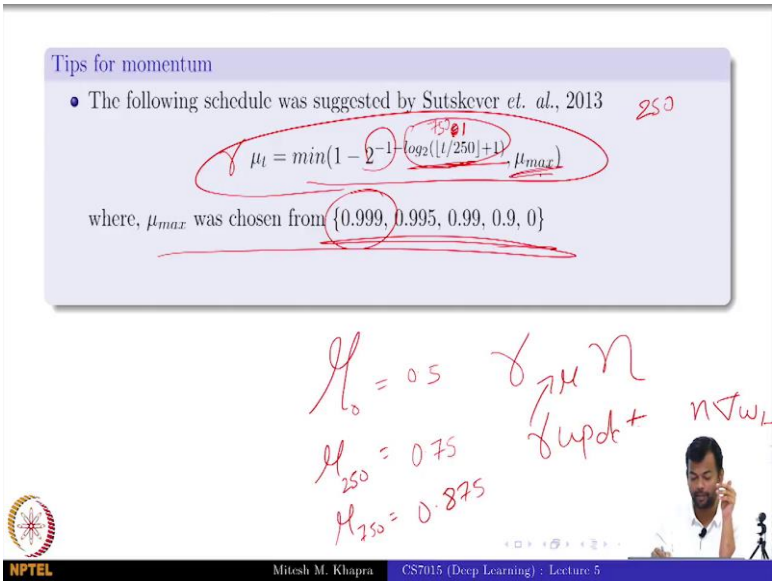
from some other point; that means, you started from some other error location right and run this algorithm again, and see if you reach a different minima.

So, the only thing you, the way you counter this is, you just try different stochastic things right, should try to start with 10 different initializations every time reach a minima and then at the end select the lowest possible of these. Did this make sense to most of you, how many of you got this oh cool I thought I was just rambling, but yeah fine. Does that make sense, to you at least ok, does it fine

Yes a local minima is a severe problem in lot of deep learning optimization and typically people get away by that, by just picking up one of these minimum fine. Now the other thing is you could use exponential decay, where with each time step you just keep decreasing your learning rate. And if this case 2; that means, at every time step you are halving the learning rate, so you just get with something like this.

But the reason I do not like this is that you have one hyper parameter which is eta which you are trying to tune and now to tackle that problem you have introduced one more parameter which is k, hyper parameter which is k. So, it becomes harder to tune that now, and there is a similar thing which is 1 by t d k, where you try to use this formula to decay or learning rate. So, both of these, I typically do not use in practice I use the second one, I prefer the second one.

(Refer Slide Time: 09:11)



Now, tips for the momentum can you make sense of this, you just stare at it it looking just come back ok. Let us see what happens at t equal to 0, this becomes 0.

Student: Log 1.

Log 1 is 0, this is 2 raise to minus 1 minus 0 which is just 2 raise to minus 1 which is 0.5. So, what is your mu t at t equal to 0.5. Does that make sense? Is it fine with everyone or is it confusing; no ok mu max is typically this. let us assume mu max.

Now, what happens at time step 250? this is 250 by 250, so this becomes 1, 1 plus 1 is 2, the best thing that you learn in this course log of 2 is 1, so this become 2 raise to

Student: 2 raise to minus 2.

Minus 2 which is 0.25. So, what is this?

Student: 0.75.

0.75, let us do one more I had t equal to 750 1 minus 1 by 8, so that is what is going to be right ok. So, then what is happening as my time steps are increasing, what is happening to, what is happening? I am having more and more faith in the history or the current gradient. What am I increasing? Actually I have made a mistake, actually this is mu is gamma there is not we did not use mu anyway what you guys just went along. So, this is gamma actually right that was a momentum term that we had. So, as a number of time steps is increasing, my gamma is increasing; that means, I am having more and more faith in my.

Student: (Refer Time: 11:49).

No history. Learning rate is eta, momentum is gamma. So, its gamma into update t minus 1 and eta into gradient at the current time step right and here gamma is actually equal to mu. Is there any more confusion that I can add? So, when I say gamma I mean mu and so that is how it is. So, as I am increasing the number of time steps I have more and more faith in the history; that means, I do not want to now get distracted by this one update which I am making right, I want to go by the history, and I am not increasing this gamma or mu indefinitely, I am capping it by a max right. Max I will have this much faith which

is 0.999 in the history. Does that make sense? This is again just a heuristic; do not worry too much about it. So, that is how it is.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 5.8
Lecture – 05
Line Search

So, we were looking at these different variants of gradient descent. We saw that gradient descent has this problem that it finds it difficult to navigate the gentle slopes. So, we came up with tricks on momentum based gradient descent and also, Nesterov accelerated gradient descent.

The trick in momentum was that if lot of your history is telling you to move in a direction, then just continue to gain momentum in that direction. So, instead of just updating based on the current gradient, you also update based on the history, right and there we saw that this is always going to be a problem that you will end up taking u-turns and we had this analogy of how you look for directions and you just overshoot your destination and have to come back and take a u-turn and come back and so on.

So, to prevent that we realize that the update done by momentum base gradient descent is two step update. You actually the first step is based on the history and then, another step based on the gradient at the current time step, right? So, then instead of doing these two steps at one go, why not just update based on the history, see what the gradient that tells you and then, we saw this nice figure. I hope it was nice and where you saw that if you look ahead point, then you will be immediately corrected with respect to your errors. So, that was about nag and momentum.

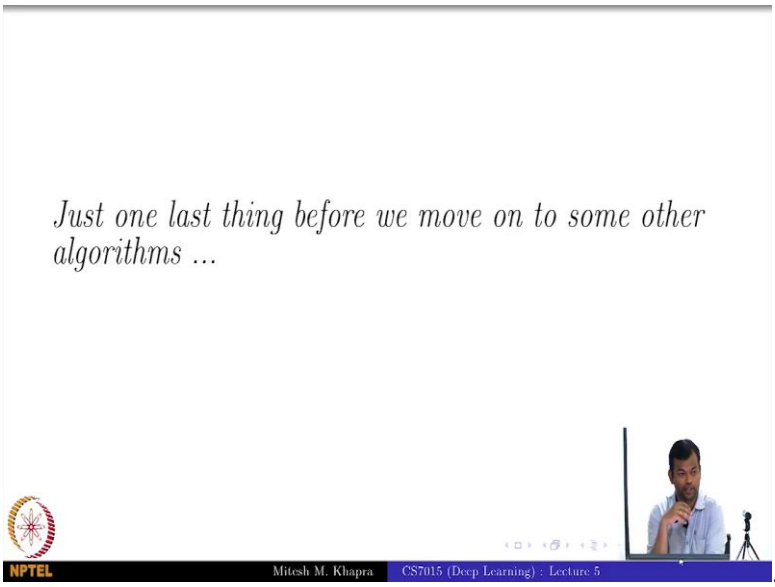
Then, we saw the stochastic versions of these algorithms, where we realize that if we do the batch version, then you go over a million points and then, make only one update which could be very slow in cases where you have large data. So, we then decided to the stochastic version where we just update for every point that again had these oscillations because we were taking greedy decisions, we were just relying on one point to tell us which was the right direction to go on and you saw that these esteem has become better as you increase the value of this k.

So, k equal to 1 is the most stochastic version and then, k equal to 2 you get the mini

batch version and then, you could just have different values of k, so that you have more reliable estimates of the gradients and in the limit if you have the entire data, then you are just doing the full batch gradient descent, right. This is the Vanilla gradient descent. Anything else did we cover, then we had some tips on the learning rate and the momentum, these are again heuristic. I gave you some ideas and you could try these in your back propagation assignment and see which one works better for you. You could see you have any peculiar observations while implement the back propagation assignment.

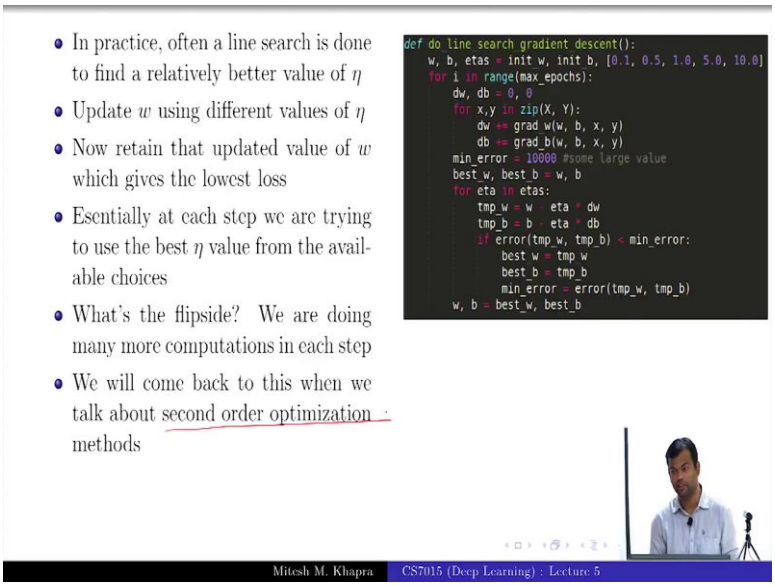
So, now there are a few more things left in this lecture. So, I will start with the line search first. So, this is one more thing before you move on to some more interesting algorithms which are the current state of the art and lot of deep learning solutions.

(Refer Slide Time: 02:44)



So, most people that you read would look at, would have algorithms that we will see after 10 minutes.

(Refer Slide Time: 02:54)



So, now this is where just to contest contextualize things, right. So, we are still trying to see, what is the light right learning rate. To use a line search is one such method where instead of just doing one learning. So, you can look at the code and just focus on this part and tell me actually what are we trying to do, how many of you get what the algorithm is trying to do. So far what we were doing is, we were just having a single learning rate, ok and we saw that this learning rate can make a lot of difference, right because if you are on the gentle part, you want larger learning rate and if you want steep part, you want smaller learning rate.

So, just fixing the learning rate to one value does not really help because then you will make, you will suffer on one of the two cases are either on the gentle case or on the steep case. Now, what line search does is, instead of just using one learning rate at every step, now whether it is vanilla gradient descent which is the batch 1 or mini batch or stochastic, right just use a bunch of learning rate. So, I have used five different learning rates here and I have computed the gradients. That part remains the same. The computation of gradients does not change.

Now, you have the value. Now, you want to be conservative, you want to multiply the gradients with this eta right, but you know that you do not always want to be conservative. In fact, in some cases when you are on the gentle slope, you do not want to be conservative at all. You want actually blow up the gradients. So, now try these

different learning rates and update w and b, ok. So, if you have five learning rates, you will get five different updated values for w, b.

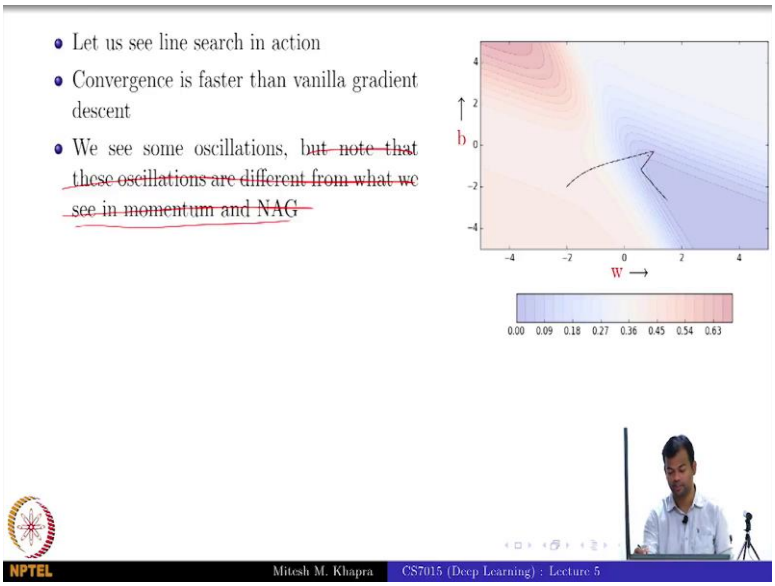
Now, plug in all these w, b values into your loss function, right and see whichever is the minimum, retain that w, b value and repeat the process. That means again you will compute the gradients with respect to this new value of w, b and the new loss function. Again try out these five different learning rates and continue. Everyone gets that.

So, now are we using a fixed learning rate at every step? No and now do you see that if we are at a gentle slope, it would pick probably this as the learning rate and if we are on steep slope which should probably pick one of these as the learning rate and even lesser than that. If you have the disruption, it does not make sense. You see the advantage of this now. You are in some way heuristically trying to adapt to the slope of the error surface, right by just giving a different learning rates. So, try all of these and whichever works best, pick it up, ok. So, that is about it. We are trying different values.

Now, what is the flip side of this? Now, if you have k different learning rates that you are trying, then at every step you have now increased your computation k times. So, earlier you just add one learning rate u just going by that, but now I have k. So, now this is again a trade off which is you have to see. Now, I will give an example where this trade off clearly works.

So, now if you are at the gentle slope, now making k more computations and moving out of that slope is definitely worthwhile as compared to just sticking to that slope where even after hundred more computations, you will not really move out of that slope. So, remember that gradient descent algorithm that we have seen where you just stick to the gentle slope after hundred iterations also right, but instead if I tried five different learning rates and there is a high chance that, I could have moved out of the gentle slope. Does that make sense? You see the advantage of this.

(Refer Slide Time: 06:31)



So, this is something that I have to talk about when I back to second order optimization So, I will see when to teach that. So, let us see line search in action. So, this is again gradient descent, this black curve, which is visible there. This is the one I am talking about which is run for few iterations and is just stuck on the steep curve. You know this story now and it is just get stuck there.

Now, let us see what happens if I run. So, now I will start running the line search based gradients descent. So, what do you expect now? So, it will just move very fast, right? So, on the first step itself, it is crossed wherever gradient descent was stuck after 50 iteration or so, I will keep moving fast.

Now, here is an interesting question. Would you see oscillations here? So, when you see oscillations? It is when your loss is actually increased from whatever it was currently. Will that happen in line search? The answer is always no.

It could happen, when could this happen? So, it depends on the learning rates that you have chosen, right. So, if you have chosen the learning rates, so suppose at one point to really be effective, you needed the learning rate to be 0.01, ok and now if 0.01 learning rate was not in your set, right that means, everything that is there in your set is faster than 0.01, so that it will again have the same problem as momentum because you will move faster than what you should actually move. So, it depends on this careful choice of the learning rate set.

So, that is all I have to say. So, there is a slight convergence would be faster than vanilla gradient descent. That is obvious and we see some oscillations ok and the statement is actually wrong. We need to remove that, ok. We see some oscillations and these could be the similar wants to the once with that we see in momentum because we overshoot because we have not chosen the right set of learning rates.

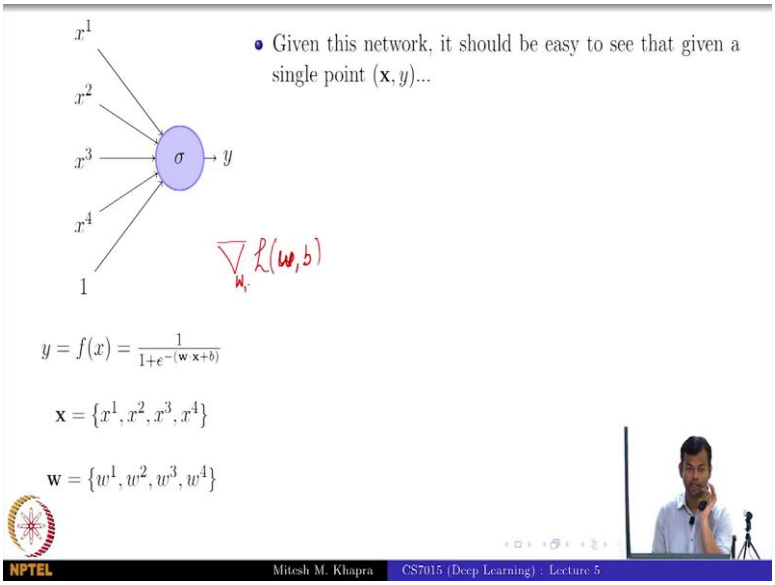
One of the learning rates which was actually needed at a particular point right say at this point. Suppose I needed to move very slowly and that very slowly say 0.001 and that was not in my set, then any other learning rate is always going to be much faster then. So, you could see oscillation.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 5.9
Lecture – 05
Gradient Descent with Adaptive Learning Rate

In this module we look at Gradient Descent with Adaptive Learning Rate. So, first we will see motivation or intuition for why we need this and once you get the motivation, I believe the rest should be straightforward.

(Refer Slide Time: 00:31)



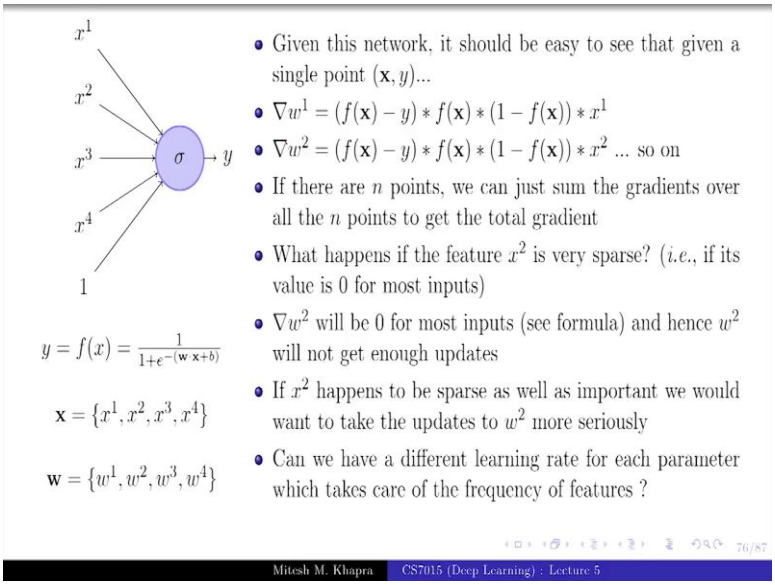
So far what we have been doing is, please pay attention on this slide. I need to define some notations and you should not get confused with that. So far we have been dealing with the situation, where we had just one feature which was x and one weight corresponding to it which was w and one bias which corresponded always on input, right? Now, we are going to look at the situation where we have more than one inputs. That means, earlier we were basing our predictions only based on the director and now, we are the director, actor, genre, IMDB ratings and so on.

So, here x 1 x 2 x 3 x 4, these are four different features or four different inputs that I have, and this is not x square. Just I know it is obvious, but I am just making it clear, right? So, this is x 1 x 2 x 3 x 4, ok. It is not probably the best choice of notation, but I

will just stick to that. So, now each of these has a corresponding w 1 w 2 w 3 w 4, ok and this is how your decision looks like. It is the dot product between the weight vector and the input vector, ok. This is how I am going to decide and that is a single sigmoid neuron again.

Now, given a single point xy, do I need to again go through this computation? Sorry w, p oh sorry, ok. I will just erase this. So, this w is actually the vector w. So, it includes w 1 w 2 w 3 w 4 and I am trying to take the derivative with one element of that vector. Do I need to show you how to compute this? Have you seen this before? Can you tell me I will show you the derivative with respect to w 1, can you tell me it will be a product of some terms? Can you tell me what is the last term going to be?

(Refer Slide Time: 02:41)



Everyone gets this? You remember this form. So, only thing which is changing is this guy, right? So, this part is exactly what we have derived and when we had one input, we just call it x and now, we have multiple inputs. So, it will depend on that particular input, right which ever w 1 corresponds to.

Now, make an interesting observation there. So, sorry before that yeah this is obvious if there are n points, we will just take the sum of the gradients with respect to the n points, ok. Now, what happens if the feature x 2 is sparse? What do I mean by that? It is mostly 0, ok. What does that mean? So, I am looking at lot of movie data, ok. Amir Khan acts in a very few movies. So, if I have a feature which says actor Amir Khan, then that is going

to be 0 for most of the movies in my data scene, right? That is what I mean by sparse.

So, if I have 10,000 movies, then probably only 50 of them would have this feature as one, ok. Does that make sense? So, it is going to be very sparse. Now, if the feature is sparse, why do we care about it? What will happen? What do we really care about when we are talking about optimization in this course, the gradients, right? That decides how well we move in the plane that we are considering w b plane or the other in the end dimensional region that we care about.

So, now if x 2 is sparse, what would happen to this? It will be 0 lot of times because x 2 is 0 lot of times, right? So, now just take a minute to understand this, right? So, now remember let us talk about stochastic gradient descent or mini batch gradient descent or even batch descent. You are going over all the 10,000 points that you have, you are computing the gradient with respect to all the parameters.

One of those parameters happens to be w 2, right? You have gone over 10,000 points, but in how many of those you will actually get the gradient for this. Only in the 15 which x 2 was present, right? Everywhere else the gradient would be 0. So, that means your sum of the gradients, overall the endpoints is going to be small or big.

Student: Small.

Small for this particular feature or for this particular weight; it is going to be small, right because you do not have enough samples where you are seeing this. So, now what would happen to the update? You started with a random value for w 2. After one epoch or making one entire pass of the data, what would happen to the updates for w 2? Very small, very few updates compare this to a feature which is dense. Do you get a lot of updates? So, you see there is something unfair happening here. If a feature is sparse, it is not getting updated enough.

Now, that was ok. In one situation if this feature was not really important, but now consider the exact example which I gave you which is this, an Amir Khan movie or not, but suppose I am doing a classification whether this movie is going to be hit or not, I would believe this feature is very important because almost always when he is the actor, the movie is a hit, right? So, you really cannot ignore this feature. You want to learn the parameters correctly for this feature. Do you get the setup right? There could be cases

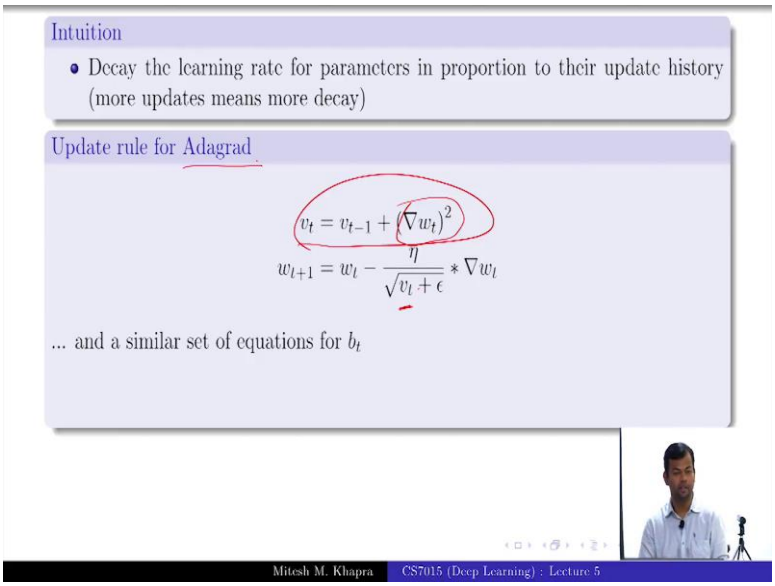
where your feature is very sparse, but at the day at the same time very predictive of the output that you are trying to learn, right and in this case, the output is whether the movie would be a hit or not.

The other example could be is Christopher Nolan, the director. So, yes probably directed less than 10 movies, but all of them have been at some point in the IMDB top 250 or something, right? So, that is a very important feature, but you will not get it very frequently in your data, right? So, you cannot really ignore these features. That means, you still want to learn these features properly. So, you have sparse features; you have dense features. We understand that for the sparse features, the updates would be slower and for the dense features, the update would be faster. The sparse would be 0 in most cases. No no. So, you will do this 0 mean thing.

No, but if it is a same value and you are going to 0 mean the data, right? So, the value even if it is 1, it is going to be very close to 0, right? So, you always assume 0 means otherwise all this does not make sense, right because if your features are not in the same range, then anyways you are in trouble, right fine? So, this is what I was trying to say that the gradient with respect to w t is going to be 0 for most inputs and hence, w t will not get enough updates and as I said if this is an important feature, we cannot really ignore it. We have to make sure that it learns better.

So, what is the case that I am making for? What do we actually need? Can you relate it to the discussion on learning rate that we have been having? So, if the feature is sparse, you know it is going to get very fewer updates. So, can we change its learning rate, so that feature gets updates a bit faster as compared to the other features. So, you get the motivation, right? How to do this is a separate story, but at least we need to do this.

(Refer Slide Time: 07:49)



So, the intuition is decay the learning rate for parameters in proportion to their update history. So, you have been recording the update history, you have been looking at the parameter, you know all the gradient w 2 that you had calculated so far, right? How many times you had computed the gradients and what those values were actually now for these sparse features? Those are going to be 0.

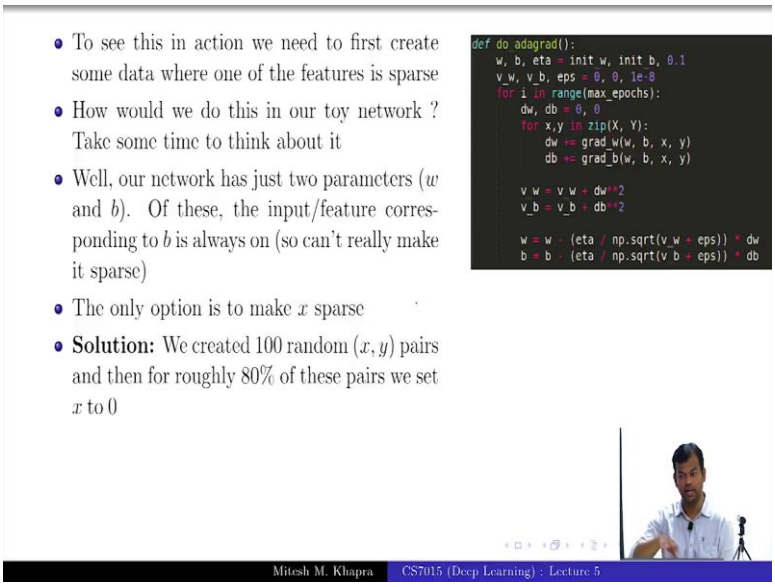
So, your cumulated history is going to be small, right? For a dense feature, it is going to be high. So, why not make the learning rate inversely proportional to this history. That means, if the feature has been updated fewer times, give it a larger learning rate. If it is not updated, if it is updated many times, give it to a smaller learning rate. Can you give me a mathematical formula for doing this? This is the intuition. Just think about it for a minute learning rate inversely proportional to update history, ok good. How many of you get that, but most of you will get it once I show you the answer.

This is my gradient which I had computed so far. I mean at this time step I will keep accumulating it in a history vector. So, at time step 0, I will take the magnitude of this again. I am taking the magnitude right because it does not matter whether you made an update in the positive direction or the negative direction, you just matters that whether how much, by how much it move. So, I will just square this quantity, so that I can get rid of the sign. So, I am taking the magnitudes and I am storing all that. So, at time step t what would vt contain? It is grad w 0 square plus w 1 square grad w 1 square and so on

up till time step t.

Now, this was my if I ignore this quantity, this was my normal gradient descent update rule. Now, do you see what I have done? I have divided the learning rate by whatever history I had accumulated. So, for the dense features what would happen is, the learning rate will increase or decrease with time, the learning rate will decrease, right and for the sparse features, relatively less. In fact, if you have written gotten 0 updates so far. So, when you have to update the first few times, you will have a very high learning rate. Does that make sense, right because this quantity would be 0. So, our eta would actually be very large. So, you see how that intuition got converted into some reasonable formula.

(Refer Slide Time: 10:13)



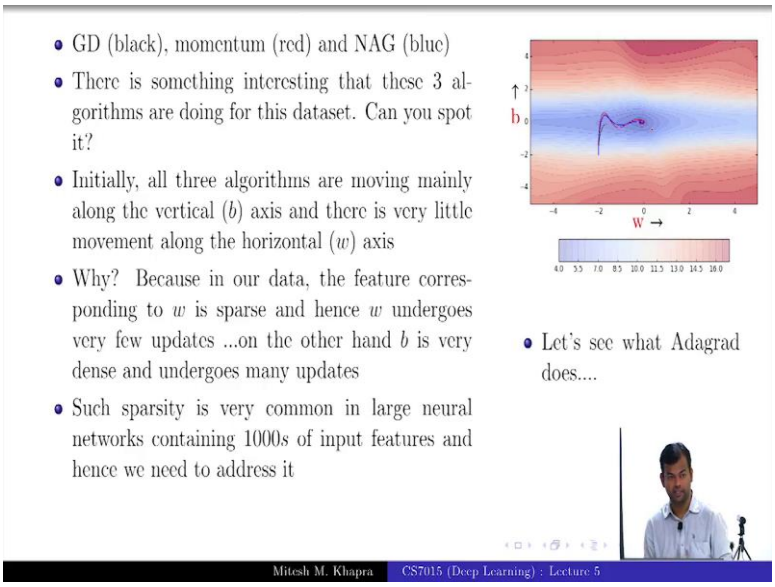
Now, can you tell me a way of actually realising this? I want to show you that what happens when you have sparse data and I want to do this with the toy example that we had where we had only one feature and other feature was always on, right? So, how do I create this sparse data? So, you should think about these because these are things you will have to do when you are practising machine learning and if you are working with the problem and you want to create some simulated data, so that you can verify some hypothesis that you have. So, how would you do this?

See I am going to create thousand data points, right which is x, y points and of course, I have this x 0 which is always on, right? So, x 0 is always on. I cannot make that sparse. What about the other feature? If I am creating thousand data points, what should I ensure

is that 80 percent of them or some 90 percent of them is always 0, right? Just as the Amir Khan case and most of the data it is going to be 0. So, what we will do is as I said we just have two parameters w and b. b cannot make sparse is always going to be on. So, what we will do is, we will make x sparse, we just create random x, y pairs and then, for 80 percent of those we will set x to 0, right? So, now this x feature is going to be very sparse.

So, now I have created some data which is sparse. One of the features is sparse and now, I want to see what happens when I run gradient descent momentum and Nesterov accelerated gradient descent and how does the algorithm behave and now, if I apply this algorithm which I did not name, it is called Adagrad, ok. This algorithm is called Adagrad if I apply this algorithm, then what how does the situation change.

(Refer Slide Time: 11:55)



So, this is what gradient descent momentum and NAG do. Now, at least the difference between momentum and NAG should be clear. NAG blue curve is inside the red curve right? So, oscillations are slightly smaller. This is how they behave.

Now, there is something very interesting that these algorithms are doing for this particular data set that I have created. Can you spot it? What is the interesting thing happening here I want you to take some time and think about and relate it to the discussion that we just had how many of you see what is happening here. Very few I will give a hint, ok. It is almost as if these algorithms went to a school where they did not

teach Pythagoras theorem. Now, related to the discussion that we just had what is happening initially. So, initially what is happening is you started from here, ok and this is the w, b planes. So, you have w on the horizontal axis and b on the vertical axis.

What is happening to all your updates initially where are you moving? You are moving along the b direction. Are you making any movements along the w direction? No. Why w was sparse? Its gradients are mostly 0. It was not being able to make any updates in the w direction or it was able to do make updates in the b direction. It did as much as it could do after reaching here it realizes that there is no point in going to be further, right? It actually took u-turn because it realise that there is nothing I cannot really go ahead. I have to now start working in a direction of w.

So, now in practice although in this toy example, it does not, it still converges fast, but in practice what will happen is you have just moved in one direction, reached a point and now from there again you are going to take right turn and reach to your destination, right? So, you are taking, you are doing something which is not fast. This is not how you would go from this point to this point. There has to be a better way, right and this is happening because w is not getting updated frequently. All the updates are initially done for b.

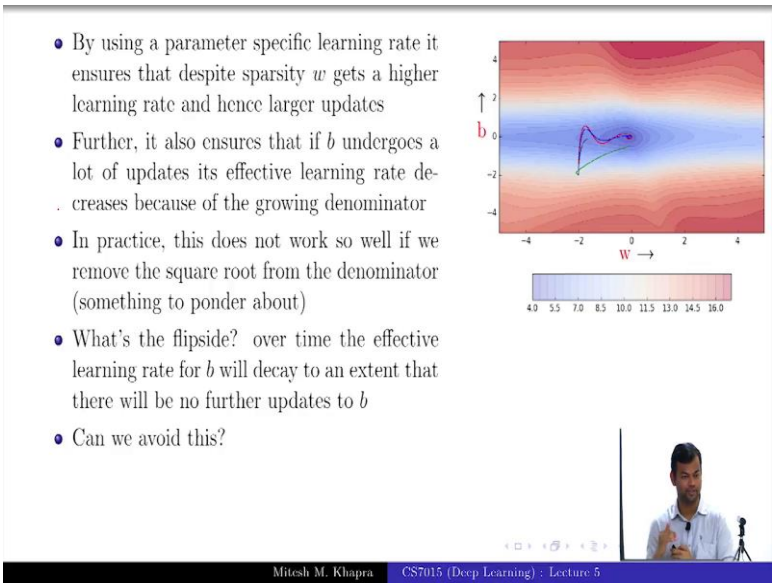
Now, when it is no longer possible to change b because you reached the optimum value for b, then only you start changing w and that to very slowly because it will have to wait for many updates to happen. For that to happen how many of you get this? So, this is exactly what is written on the slides because in our data the feature corresponding to w is sparse and hence, w undergoes very few updates and b is very dense and it undergoes a lot of a updates.

Now, such sparsity is very common in large neural networks which have thousands of features, right? So, you can imagine this. Now, if I have thousands of features, now suppose I am doing credit card fraud detection, ok now say one of my features is corresponding to some education that the person had and suppose he has done some very less sort after degree or less sort after curriculum.

So, that feature is going to be sparse where most of the cases, but I cannot ignore it. May be this is the most predictive feature that I might have, right? So, you could think of various cases where you have thousands of features out of which many are going to be

off for a given example, right? Everyone sees that this is the real world scenario where lot of your features are going to be sparse and in many cases, you cannot ignore the sparse features, ok fine. Now, let see what Adagrad does. Any guesses?

(Refer Slide Time: 15:18)



So, I am running this, we should start seeing something a green curve starting from here. Do you see what is happening expected? Now, try to guess if you are going to run into a problem. I have deliberately halted the algorithm. I just want you to think if you are going to run into a problem, ok. All of you think you have something which makes sense. So, now I have run it for in this case again this is the toy example. Hence, you do not see a lot of difference between these algorithms in terms of number of steps taken to converge, but in real world application, it would be very different, but now what has happened is I have run the algorithm for as much I can and I am then stuck here. I am not being able to move forward. Why is this happening?

Well, I am the histories accumulating it is growing. Now, what am I doing to the learning rate? I am just killing it, right? It is eta by a very large constant. Now, that is going to be very small. So, no matter how big my gradient is, it is going to get multiplied by a very small learning rate and I cannot just move any forward anymore, right? So, see that will happen. That is why in this case this is some point here which I do not want to go over now and it is this.

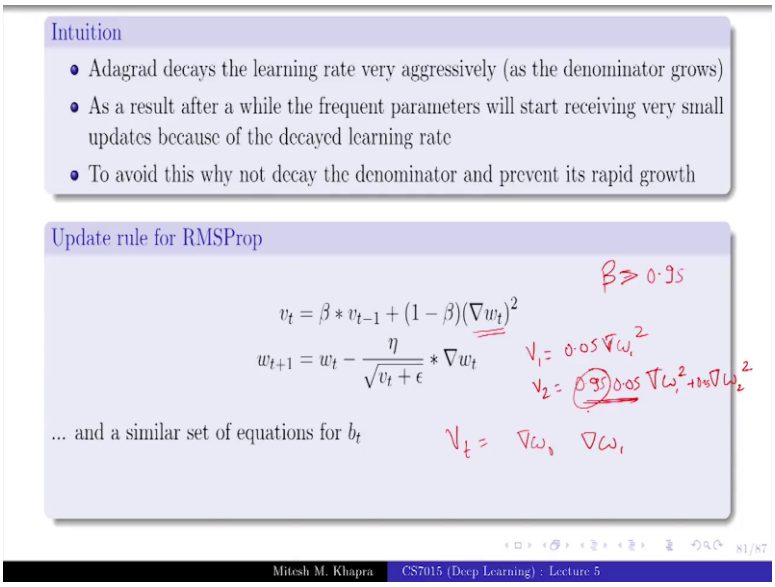
In fact, I do not have an explanation for that, but this one observation which people have

made that remember we have the square root in the denominator. If you remove the square root in principle, you are still doing the same thing, right? You are still making it inversely proportional to a cumulated history, but it does not work well when you do that. That I do not know why it happens and I just read these comments at several places that it does not work when you remove the square root from the denominator, but that is not important for this discussion. That is just point for reference later on.

So, right now what I am trying to say is that it did the right thing. It started making updates for w also and started making larger updates, hence we see this simultaneous moment in both w and b direction. But the flip side is over a period of time, the effective learning rate for b will decrease so much that we no longer be able to move in the vertical direction, right and if I am not being able to move in the vertical direction, we will not reach the minima. In this particular example not always, but in this particular example you need to move further in the direction of b, but a learning rate is not allowing you to do that. So, that is what is happening.

So, now can you avoid this? Yes. How? Multiply by, so first divide it, so that the decreases, then multiply it. So, that does not decrease. All of these are interesting ideas. I am not I mean it is very hard to say upfront whether this is wrong or right, but yeah these are you get the idea basically something is happening which is you are aggressively killing the learning rate.

(Refer Slide Time: 17:59)

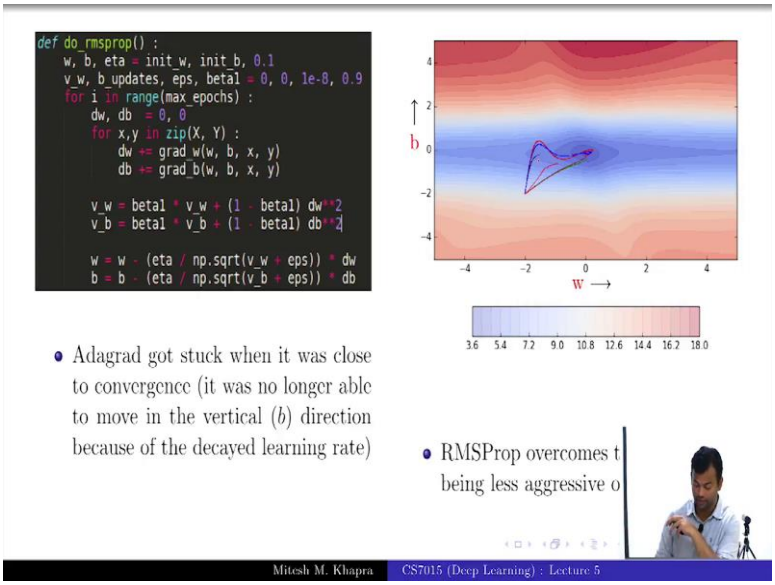


Now, I just want to make sure that you are not so aggressive. So, what happens because of the aggressive killing, is the frequent parameters, they start receiving fewer updates. Now, this is what RMSProp does. I want you to stare at this for a minute. Assume that beta is going to be something which is greater than 0.90 or 0.95 or something and try to make sense of what is happening, try to imagine what is vt is going to look like, in terms of grad w 0 grad w 1 and so on. To start from v 1 and see what happens what was v 1 earlier and what it is going to be now ok, but it still grows my magnitude when I am still adding stuff. So, how does it help me in not blowing of the denominators?

So, yeah I think you most of you get. So, again this is the trick is basically you are using this exponentially, exponential moving average. So, even at the first step earlier I was doing grad w t square. Now, actually doing 0.05 into grad w t square, oh sorry grad w 1 square, right? So, that is what my v 1 is going to be. Now, what is my v 2 going to be? It is going to be 0.95 into 0.05 grad w 1 square plus grad w 2 square, right? So, this quantity is even shrinking further and at each step this is going to keep a 0.05, ok and you see now at each step this is going to get multiplied by this quantity and shrink further.

So, now I am not aggressively growing the denominator. I am not considering the full gradient, but only a fraction of it and in fact, a very small multiple of it. So, I am still accumulating the history, but I am not being very aggressive while doing that, right? So, you understand this. Everyone gets this.

(Refer Slide Time: 19:54)

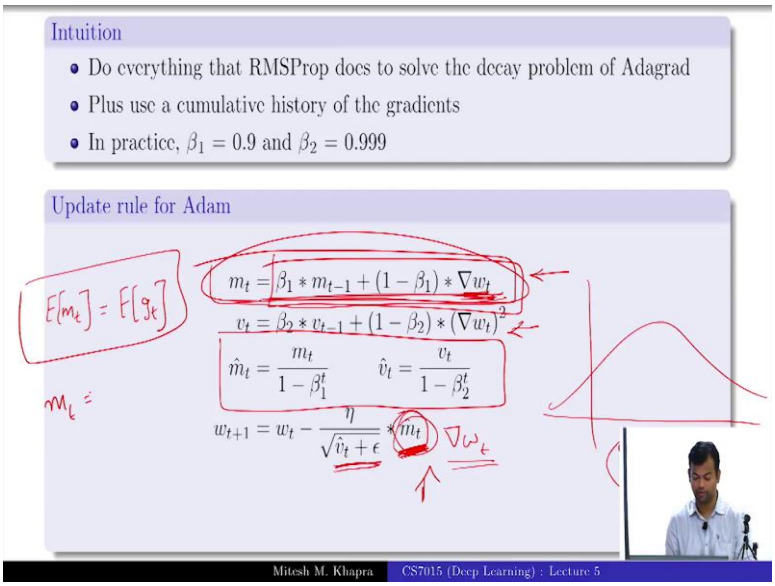


So, now let us see if we run what would happen. Any guesses, ok? So, initially now this is I think a brown curve. It is already there, but you can see it. So, I will keep running it and at some point, it will diverge from the green curve. Yeah do you see that? Now, I have reached its destination, right; so, at the point where the b learning rate, the learning rate for b was getting killed. In this case that does not happen because you have prevented the denominator from growing very large actually multiplied by its small values, so that it does not grow very fast.

So, Adagrad got stuck when it was close to convergence because the learning rate was killed and it was no longer able to move in a direction of b, but for RMSProp, it overcomes this problem by not growing the denominator very aggressively, ok. Now, can you think of any further modifications? There is everything that you learned so far and my everything yeah.

Yeah I am not very sure why that I agree that I am also bit surprised that it completely overlaps with it. I checked it and that is how it turns out to be and guessing it is an artifact of the artificial data that I have created. So, it is trying to say is actually making sense that it should not overlap so much, right? Initially it should slightly be biased towards b and then, probably that is what you are trying to say right, but I told it just an artifact of this data that I have, but what matters is from as going to say illusion, but from the illustration is that it actually does not kill the learning rate.

(Refer Slide Time: 21:48)



What is the one idea that now think of everything that you learned in starting from gradient descent, then you tried to improve it using something, then you tried to further improve it and so on and now, we have taken a slide d 2. From there you are now focusing on the learning rates, but there were other things which you are doing earlier. Can you bring those back, add momentum. How many of you say add momentum as if I can just added. You are right actually.

So, let us see what we can do. So, it does everything that RMSProp does. That means, it tries to make the learning rate inversely proportional to a sane cumulated history. By sane mean it does not allow the history to blow up and it also will use the cumulative history of the gradients. So, let us see the update tool for Adam. So, what is this term doing? Actually it is taking a moving average of; there is the same as the momentum base role, right? Just taking a moving average of your gradients, ok. The same analogy that I am going to phoenix market city, I am just taking all my history into account, ok and vt is again a cumulative history. This is the same as what was happening in RMSProp, right where you get lost.

Now, what would be the next step be? Can you give me the final update rule? At least think about it. mt into vt, no. Ok just try to think about it and it is very hard to say it out there are too many grads and suffixes and so on. So, just think about what you did in the momentum case, ok. Now, there is one more step which I am going to ignore. I will just

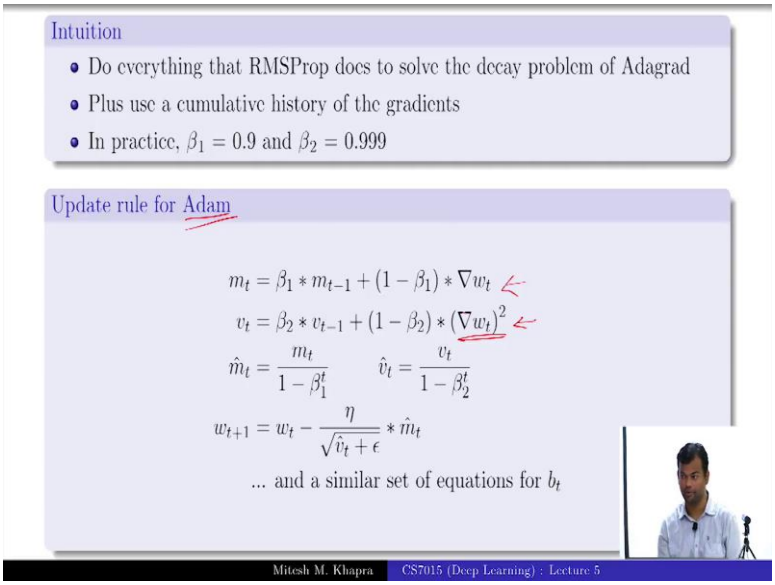
say what that step is and then, I will come back to that later on.

So, this is something known as bias correction, ok. Just ignore it for the time being. I will come back to this discussion. Just for the time being just assume that I am taking mt and dividing it by some quantity, right? So, for all practical purposes I am just using mt, just dividing it by a quantity, ok just for now. That should suffice and then, my final update rule is going to be this.

So, let me go over this. What did you expect here in a normal gradient descent? They should have been grad w t. That means the derivative with respect to current w, ok. Instead of that I am using a cumulated history. Instead of using just this quantity, I am using a cumulated history. Does it make sense? This is same as momentum base gradient descent. How many of you get that? Ok and now, this quantity there is nothing new. This is the same as what RMSProp suggested that you divide the learning rate by a cumulated history of gradients, right? So, just a combination of these two, one is take care of the learning rate and the other is use a cumulative history. Does it make sense now? Ok, fine.

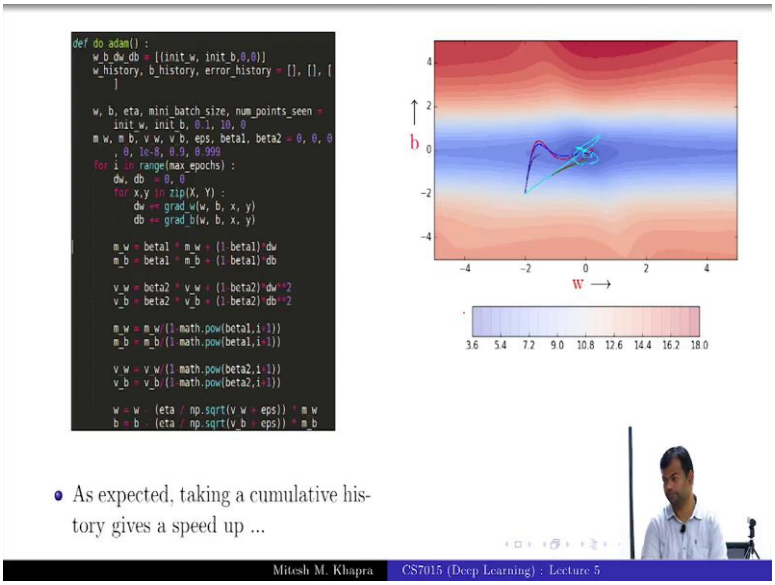
Now, this part is something that I need to tell you about. So, I will tell it to you after I run the algorithm and then, I will come back to that, but is the update rule clear that it is a combination of momentum plus killing the learning rate ok fine.

(Refer Slide Time: 25:19)



It is a similar set of equations for bt.

(Refer Slide Time: 25:21)



Now, let us see what happens to this algorithm is actually call at Adam. It stands for Adaptive Moments, right? Yeah what is can you tell me why that name?

Why moments?

Student: Sir, mean is.

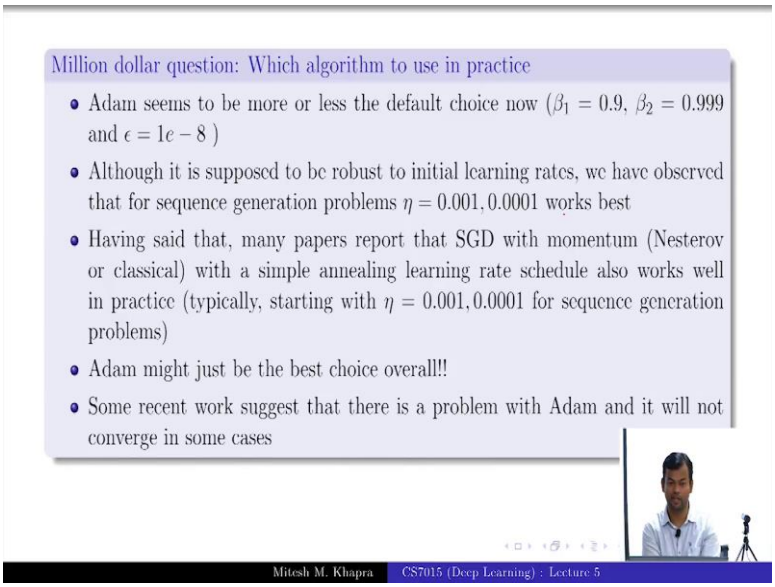
Good, where is the mean? Here this is a mean. This is a moving exponentially weighted average, right? This is an exponentially weighted mean. What about this? What is this quantity? If you take the average of this is the second moment, right exponentially weighted second moment, right? So, using the first moment and the second moment we come up with an adaptive learning rate.

So, now I will run this algorithm. Are you able to see this? See a coloured curve ok. So, it is here you see that now ok do you see what happen? Do you see this curve? Everyone sees that, ok. So, what is happening, it is taking u-turns, right? So, again whatever happens because of momentum, it is happening in this case also and then, finally it will converge again. Let me be clear that in this case now it should be very clear. We need to change, who is TA for the slide. So, this colour needs to be changed or it should be bright right from the first. So, what is happening is it is getting overlaid and then, it becomes bright when we need to have a brighter colour right from the beginning, ok.

So, this again in this toy example, right; You do not really see the speed as such because

all of them are converging you know almost the same number of steps, but this again I repeat for the toy example, but at least you see that the behaviour is very different and behaviour is consistent with whatever you have put into the update rule, right? In one case the learning rate gets killed, in the second case it does not decay and in third case when you using this moments, sorry this momentum term you again have this behaviour similar to the momentum gradient descent, where you actually overshoot and then, you come back, ok. So, is that clear all these algorithms? Ok. Now, here is the million dollar question.

(Refer Slide Time: 27:53)



Which of these two you use in practice? So, what are the options that you have for your back propagation assignment? Even if you have not read the assignment, you should just tell me based on whatever you have learned you have gradient descent.

Student: Momentum.

Momentum.

Nag RMSProp.

Student: Adagrad.

Adagrad Adam, ok. So, which of these would you choose and if there is one or which is called eve, but it did not really gain much momentum, but Adam. So, in practice Adam

seems to be more or less the default choice. I should tell you that recently there was a paper or called couple of papers which actually show that there is a slight error I mean there is you could showcase where Adam will not actually converge as expected with, but still then after that as is the case in whole of deep learning resources that one person says this work and immediately the next is someone else. This does not work or vice versa, right?

So, someone show that this does not work. Adam does not work in some cases, but then someone else did detailed study showing that in most practical applications, ok you have taken a toy data set where you can show something under some conditions. Adam will not converge, but if I look at real world data sets like MNIST, image data or something, those conditions do not hold there. So, Adam really works well. So, in practice Adam is more or less the standard choice. Nowadays at least all the image classification work which deals with convolutional neural networks and convolutional neural networks and so on, that uses Adam as the optimization algorithm.

We have used it largely for a lot of sequence to sequenced learning problems and it works well. Although it is supposed to be robust to the initial learning rate, right because you are tampering with the learning rate as you go along, right? You are not sticking to eta, but you are conveniently blowing it up or shrinking it based on your requirement. So, it should not be sensitive to the initial learning rate, but we have observed that at least for the sequence generation problems if you use one of these learning rates as a starting point, they work best of course. Of course these are heuristic, right? We also depends on how much data you have and so on.

If you are going to train, but only thousand samples and first of all of course you should question why are you using deep learning, but you have gone pass that question already. Has everyone else has, then you are still be using a deep neural network and in that case may be these learning rates are going to be very small, but in general for a large number of data sets out there which lot of academic research happens which are of reasonable size, these learning rates happen to be well in practice.

Now, having said that many papers report that SGD with momentum either the Nesterov momentum or the Vanilla momentum with a simple annealing learning rate; We remember we did this learning rate decay either a constant decay or that heuristic decay

that after you look at the validation loss and then, decide whether to decay or not. That also seems to work at par with Adam, right? So, my advice would be that if you really know what you are doing with SGD and momentum right, that means if you really know how to look at the loss, how to track it, how to adjust the learning rates and so on.

With a little bit of manual tampering, it should work as well as Adam, there are people which show that it works well as Adam, but if you are just a practitioner who does not really want to bother too much about setting the learning rate, setting the momentum, setting the schedules on both of them, remember for momentum also we had a schedule and was just given by one of these papers and it might differ for your application. You might want to tweak that a bit. So, if you are not really bothered about doing all these things, then Adam would just be over all the best choice, right with very minimum tempering of the initial learning rate.

As I said some recent work suggested there is a problem with Adam and we will not converge in some cases, but then it still I mean I would say that juries not out on that yet because there is of course theoretical angle to it and also, the practical angle. Again practice has been used widely for the last 3 to 4 years. At least and it works well in a large number of applications, right? So, that is why Adam would typically be the overall best choice.

Now, there is this one thing which I need to do which is I need to tell you why do we use this bias correction? So, now what do you actually want to you are taking a mean, ok. You do not want to rely on the current estimate of the gradient, but you want to take an exponentially moving average of the gradients.

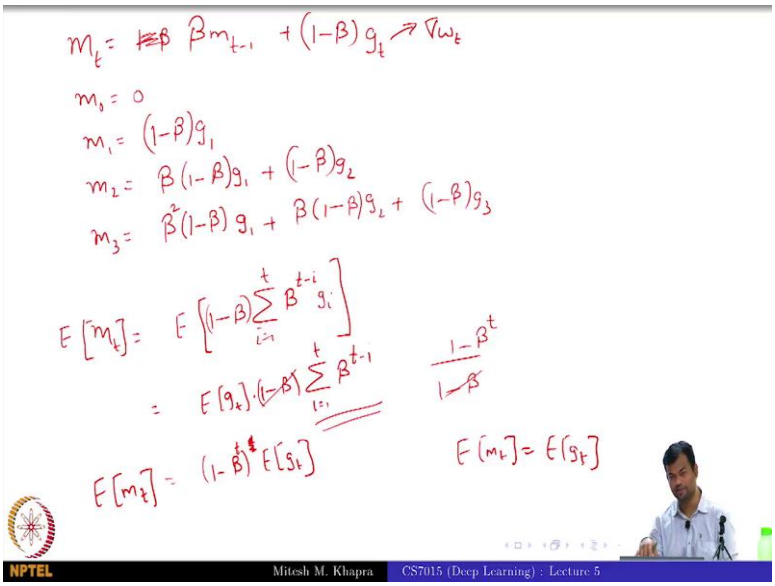
Now, what would you actually would be doing all this, what is the intuition behind this since you are talking about moments and so on? Can you think in terms of probability distributions? So let me just try to say this we write that your gradients, your values of grad wt, right and I will just I think alternately use gt instead of grad wt, just needs to gradient in that form. It actually comes from some distribution depending on the point at which you are, right? The gradient would change, but it comes from a certain distribution and now, what you actually want at any time step when you are making this update, this particular update ok. Is it clear? Yeah when you are making this update, what would you actually want? It should not move too much away from sorry.

So, now your gradients how you are computing say if you are doing the stochastic version, you are computing it for every point that you have, right? With respect to that point you would have some loss function and some derivative with respect to your parameters. If you move on to different point, you will have some different parameters. So, there is some randomness in this, ok. So, I am saying that these gradients would be treated as random variables which can take on values according to a certain distribution.

And Now, what do I mean? So, what would I actually want when I am making an update. So, I have to one basic choices. I could have just use grad wt which is the derivative with respect to the current time step. Add the current time step, ok. Instead of that I know why I am not happy with that because it has this problem that it could pull me to the extreme. So, at this point is actually saying change it, change your w value in a particular way which is more suited to me some other point would say something else. So, what we want is that whatever update we make should be very close to the dash of the distribution mean of the distribution, right and instead of computing the mean, we are computing a moving average and exponentially moving average.

So, now what do we actually want to say I said that gt is the random variable for denoting the gradient. What do I actually want? I want the expected value of mt should be equal to what the true expected value of gt. This is what I want because I want to I do not want my updates to move in the extreme. It should be closer to the average to the mean of the distribution. Do you agree that this is my wish list? This is what something that I should desire for, ok. Now, let see what is mt actually if I want to write it as a formula.

(Refer Slide Time: 35:28)



So, I have mt is equal to 1 minus beta. I will call this as gt, right? So, remember the gt is grad of w t, ok. So, now let us try to write formula for this. So, m 0, I will set it to 0. So, m 1 is going to be 1 minus beta in to g1, ok. M2 is going to be beta into 1 minus beta into g 1 plus 1 minus beta into g 2 and m 3 is going to be beta into 1 minus beta square g 1 plus beta into 1 minus.

Student: Beta square.

Sorry beta square.

Student: Minus beta.

Minus beta, wait is the first term correct.

Student: Yes.

No beta, ok. So, wait what am I? Oh beta is getting multiplied, to g 2 plus 1 minus beta into g 3. So, what is the general formula going to be? It is mt is equal to, so 1 minus beta can come out, ok. Summation i is equal to 1 to t 1 minus beta

Student: Beta square.

beta raise to t minus i and gi, right ok. So, this is what my mt is, ok. Now, let me take the expectation of this. This fine Now, ok this is b 1 minus beta. Now, this is going to be is

that fine. So, what is this? This is an ap gp. What is the sum going to be?

So, it is going to be 1 over 1 minus. Oh it is actually sorry 1 minus beta raise to t over 1 minus beta. Is that fine? So, what will happen is, this will get cancelled and what you are left with this 1 minus beta raise to t into e of gt, ok. So, what is the relation that you have E of mt, ok. E of mt is equal to 1 minus beta raise to t into E of gt. What did you actually want?

Student: E of gt.

Right so, now how will you ensure that divide by divide mt by 1 minus beta raise to t and that exactly the bias correction that we have done, ok. Sorry about this messy derivation, but I guess most of you get it. If not, we will just type it properly and upload it in the slides. How many of you got this? Most of you got, fine; so, that is the similar derivation for vt also, fine. So, that is why we need the bias correction.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

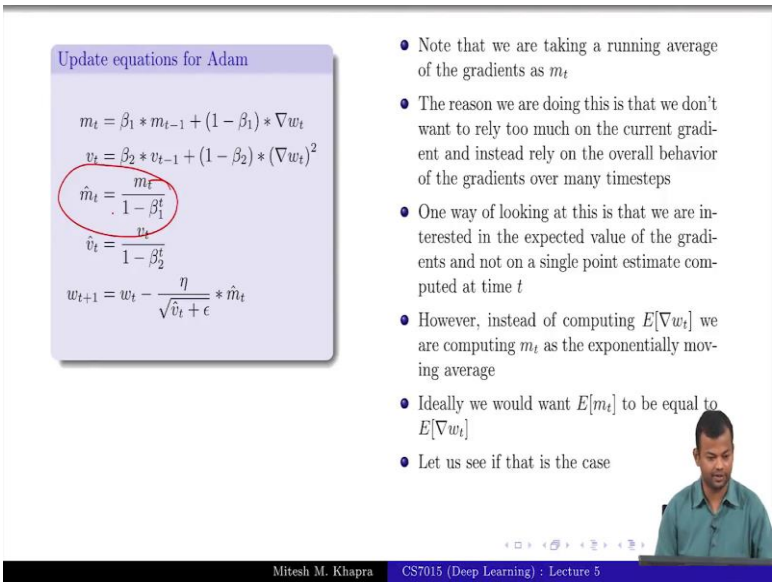
Lecture - 05
Explanation for why we need bias correction in Adam

(Refer Slide Time: 00:12)



So, in this video we will try to look at an explanation for why we need Bias Correction in Adam, or in other words I want to explain why do I do this particular step why did I take m t and v t as it is, but why did I do this particular step which I called as the bias correction step.

(Refer Slide Time: 00:21)



So note that in the case of Adam if you look at this equation for m t we are actually taking a running average of the gradients and storing it as m t right. So this is the gradient and we are taking a running average or exponential running average of these gradients, exponentially decaying running average.

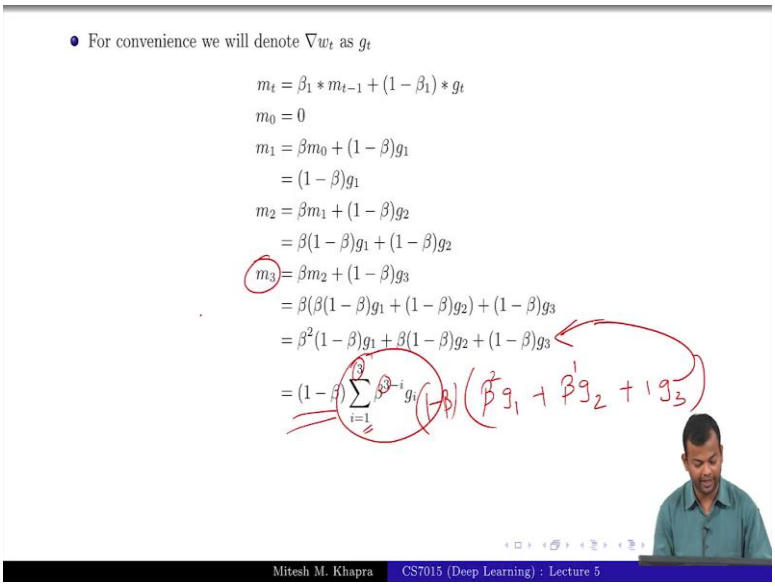
So, the reason we are doing that is that we do not want to rely on a single estimate, so we do not want to rely only on gradient of w t we want to look at the overall behaviour of the gradients over multiple time steps and then take a decision. So that means, in one particular gradient at time t is actually pushing us in some direction we do not want to be very hasty and start moving there we want to accumulate the history and appropriately weigh everything in the history, that is the idea behind taking this running average of radiance.

And the other way of looking at is that we are interested in the expected value of the gradients and not the point estimate at time w t right? At time t rather so gradient of wt which is this quantity which is the point estimate at time t, we are not interested in that were interested in the expected value and our behaviour should be according to the expected value that is what we desire.

So however, instead of computing the expected value of this quantity which should have been ideal, we are computing mt as the exponentially moving average. So in the ideal case we would want that these two quantities are the same that the expected value of mt

the way I am computing it and the expected value of the gradient of w t should be the same. If that is the same then I am fine because then; that means, I am just taking the expected value or the, of the gradient instead of relying on the point estimate ok. So, let us see if that is indeed the case.

(Refer Slide Time: 02:14)



So, for convenience we are going to just denote this gradient w t as g t because it is cumbersome to write this grad symbol and we will just not make it so readable the derivation that we are going to do. So I am just going to replace that as g t so what I have written is g t here instead of grad w t right. So from now on I will just use g t for grad w t is that fine ok, so we have this expression for m t.

So, now let us just try to expand it and see what happens right so m 0 it is going to be 0 because that is my starting points I have no history nothings, so I will just going to keep it as 0, m 1 is my first time step at which it is going to be beta into m 0 so I am just substituted t minus 1 and t here. And in the original expression I have just substituted appropriate quantities for m of t minus 1 and g of t, so m of t minus 1 is 0 m 0 and g of t is g 1 and of course, b 0 m 0 itself was 0, so what will be left it is 1 minus beta g 1.

Now, let us look at what happens is m 2, m 2 is going to be beta m 1 plus 1 minus beta g 2, but I already have an expression for m 1, so I am just going to substitute that here and this is what I get. Now let us look at m 3, m 3 is again going to be beta times m 2 plus 1

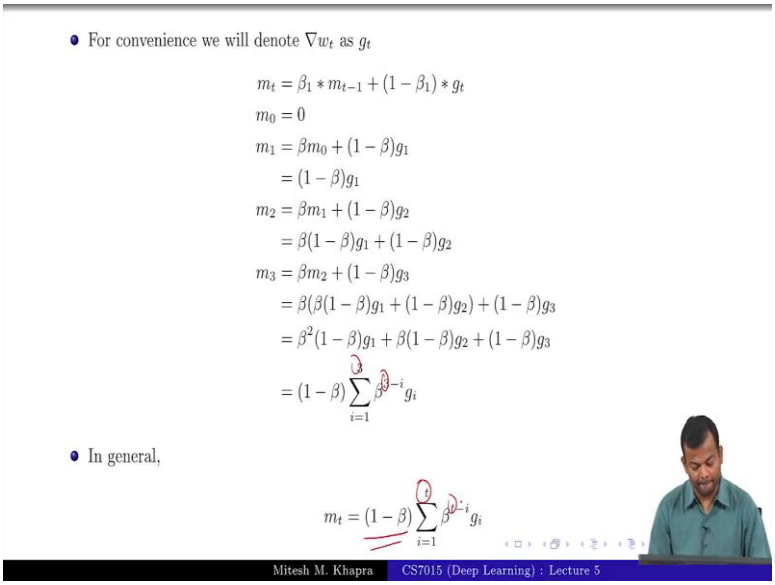
minus beta times g 3 and I have an expression for m 2 so I am going to substitute that here and see if that leads to something interesting.

So, I have just substituted the value of m 2 here right and I already had the m 3 part here the, this term here as it is ok. And now let us see so this already starts looking something interesting you see some pattern here, in particular we could take these 1 minus beta terms outside they can be taken common and then you will be left with beta square g 1 plus beta square g 1 plus beta g 2 plus g 3. So let us try to write this more compactly right, so I have taken one minus beta common and then I have written the remaining terms as this particular summation and you can verify.

So, when i is equal to 1 this is going to be beta 3 minus 1 which is beta square into g 1. When i is equal to 2 this is going to be beta 3 minus 2 which is going to be beta into g 2 and when i is going to be 3 this is going to be beta raise to 3 minus 3 which is beta raise to 0 which is just 1 into g 3 right. So we get back the same expression that we had here of course, there is a 1 minus beta outside, so this is a more compact way of writing it and this was for the 3th entry right this was for m 3, the third entry.

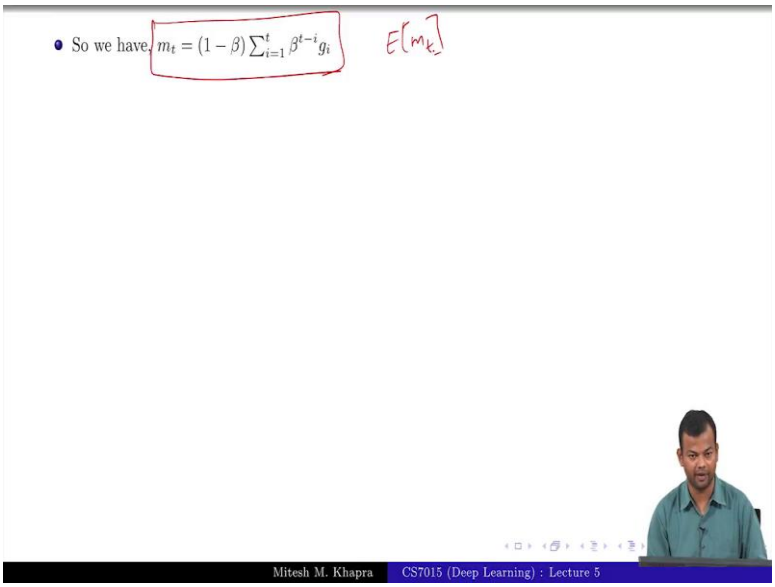
Now, what if we want to write it for the t'th entry in general, what if we want to write the expression for m t.

(Refer Slide Time: 05:05)



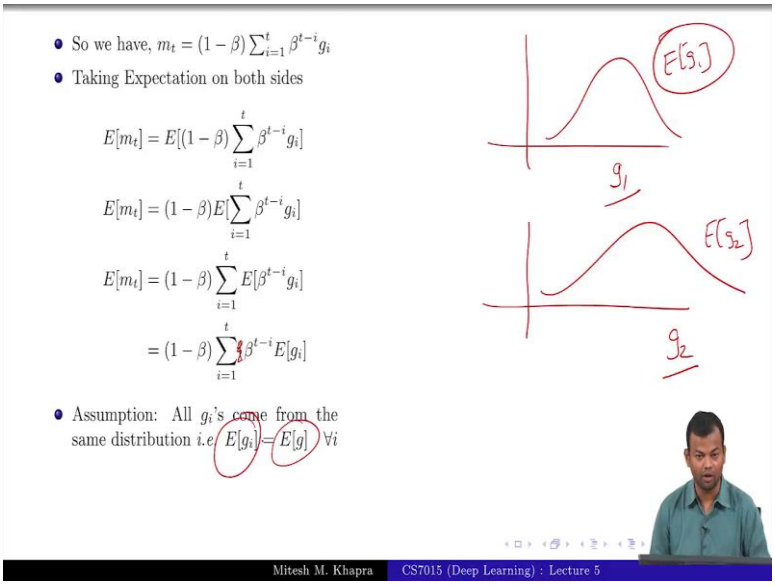
So, in general m t we can write it as 1 minus beta as i equal to 1 to t b beta t beta raised to t minus i into g i right. So this 3 is here I have just replaced them by t s right you can just verify that this is from you can just generalize from the third entry to the t'th entry.

(Refer Slide Time: 05:27)



So now, let us see we have the following expression we have simplified the expression for m t and written it more compactly, but what we were eventually interested in the expected value of m t right, we wanted to show that certain things holds for the expected value of m t.

(Refer Slide Time: 05:40)



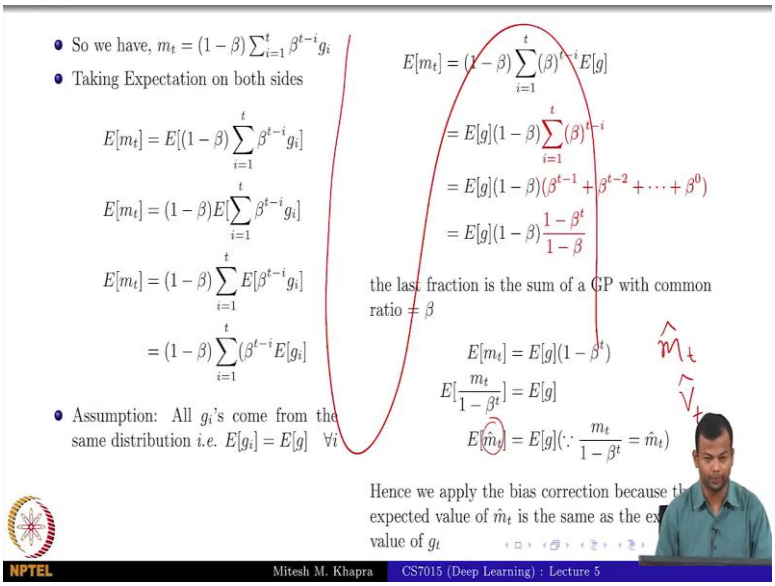
So, you just take expectation on both sides so this is what we will get ok. Now 1 minus beta is of course, a constant so I can move it outside the expectation, so then I get an expectation of a sum.

Now, the expectation of a sum is the same as the sum of expectations, so I can write it as a sum of expectations ok. Now again beta is a constant so I can take it outside the expect expectation, so what I will be left with is beta raise to t minus i outside and expectation of g i right. So this is actually expectation of g 1 when i equal to 1, then expectation of g 2, expectation of g 3 and so on.

Now, we will make an assumption that all these gi's; that means, the gradient at time step 1, the gradient at time step 2, the gradient as time step 3 and so on they all come from the same distribution ok. We are going to make that assumption so let us try to understand the implication of that right? So let us say this was a distribution from which g 1 came right suppose I am dealing with a scalar quantity and maybe this was the distribution from which g 1 came. Now g 2 could have come from a different distribution, g 3 could have come from a different distribution and if that was the case then expectation of g 1 would be different from the expectation of g 2 and so on.

So, what we have assumed to it will make things simple for us is that g 1, g 2, g 3 any g i comes from the same distribution and hence you can say that the expectation of all these gi's is going to be just the expectation of g, that is this one single distribution from these which these entries come this of course, a very strong assumption, but we are going to live with this assumption.

(Refer Slide Time: 07:16)



So then this expectation of g i just becomes expectation of g, so I have gotten rid of the index i; that means, I can move it outside the summation right so this is what I will get now. These two have come out of the summation and inside I have this quantity, now let me just expand this quantity this is nothing but beta raise to t minus 1 plus beta raise to t minus 2 plus so on at last you will reach t minus t which is just going to be beta raise to 0.

So, this is nothing but a sum of a g p with common ratio beta and I can replace that sum by this formula, you know this is the formula for the sum of a g p with common ratio beta. So I have just replaced that and now what happens is this 1 minus beta and 1 minus beta cancel out, so I get this particular expression that the expected value of m t is equal to the expected value of g into 1 minus beta t.

So, I will just take 1 minus beta t on the other side and I can move it inside the expectation because it is a constant it does not matter. So I will get as oh actually yeah I can just move it inside so I will get it as expectation of m t over 1 minus beta is equal to expectation of g t right and this quantity the one which I have circled is nothing but m hat t right this was exactly the bias correction that I was applying. If I go back to the previous slide or the slide before that, so this was exactly the bias correction that I was applying.

So, what I have inside is this, so what I have shown is that if I apply the bias correction then the expected value of the bias corrected m t is equal to the expected value of the gradient and that is actually what I wanted, I wanted that whatever m t. I am computing if I look at its expected value it should be the same as the expected value of my gradients and that is what I have arrived it.

Hence this bias correction makes sense and hence we apply this bias correction for Adam. So this we have shown for m t, we had a similar expression for v t right, so for m t we had this bias correction as m hat t and similarly for v t also we had this bias correction as v hat t so you can derive the same kind of derivation for v t also and show that that bias correction makes sense right? So this is an explanation for why you do bias correction in the case of Adam..

Thank you.